[Day 8 종합실습-4] ecommerce 매출 분석 및 성능 튜닝

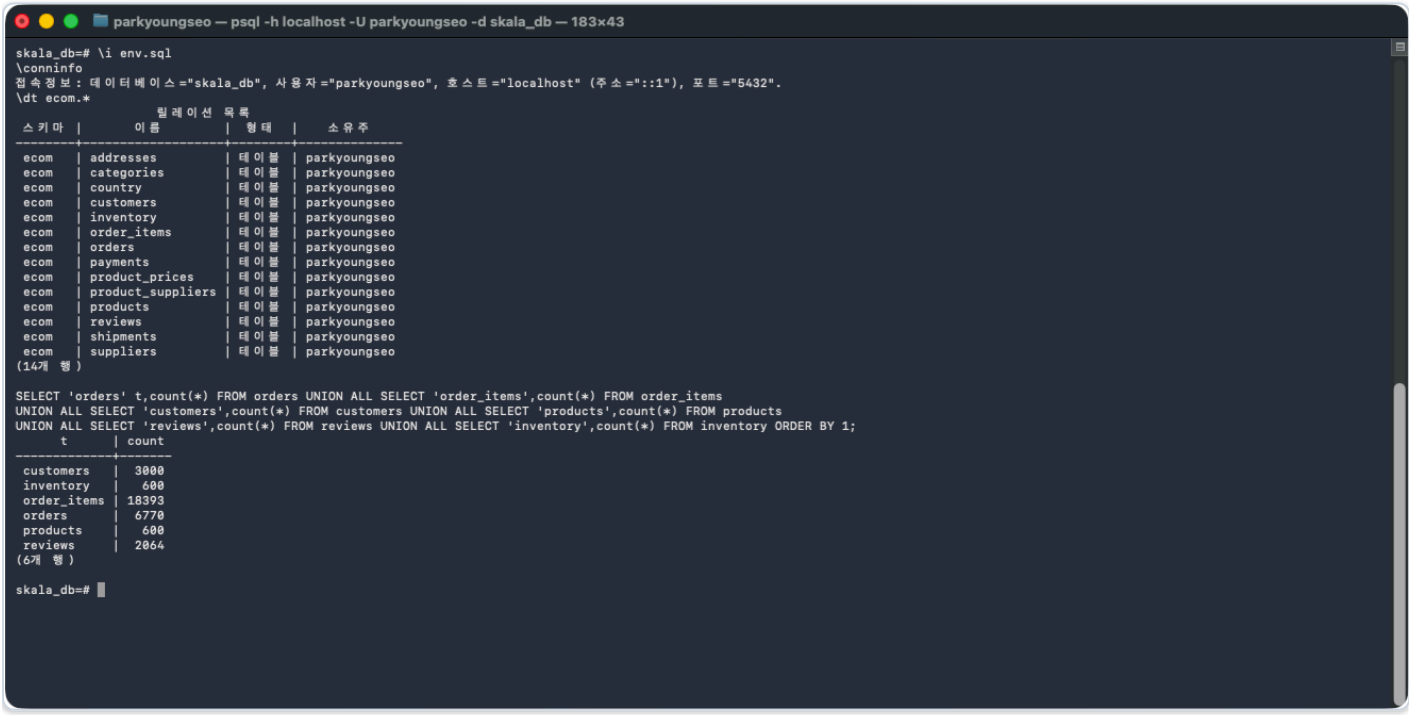
AI 서비스를 위한 SW 기초 Full-stack Engineering — 스마트 데이터 이해 및 활용 | 종합실습-4 | 광주캠퍼스 4반 박영서 | 2026-07-24

실습 결과 요약

- 실행환경: **PostgreSQL 17.10** — DB **skala_db** · 스키마 **ecom** · EXPLAIN (ANALYZE, BUFFERS)로 계획·수행시간 검증
- 데이터: customers 3,000 · products 600 · **orders 6,770** · **order_items 18,393** · reviews 2,064 · inventory 600
- **제출물 1**: Q1~Q11 각 문항 **튜닝 전/후 쿼리·실행계획·결과** (병목 → 튜닝 기법 → 개선 검증)
- **제출물 2**: 3가지 조인 알고리즘(Nested Loop / Hash / Merge) 실행계획·특성 비교
- **제출물 3**: Materialized View(mv_daily_gmv) 생성·갱신·리포트 가속(약 860배) 및 갱신 전략
- 튜닝 기법: 부분·복합·커버링 인덱스, 쿼리 재작성(count(DISTINCT) 제거·불필요 Sort 제거), 비용기반 옵티마이저 크로스오버 분석
- 채점 기준 준수 — 요구사항 1:1 반영 · 필요 컬럼만 선택(`SELECT *` 미사용) · DB 프로그래밍 **주석 처리**(모든 SQL 파일에 병목·튜닝 근거 주석)

측정 방법 및 유의점 — 본 데이터는 소규모(orders 6,770·order_items 18,393)라 절대 실행시간이 수 ms대이며, EXPLAIN ANALYZE의 행별 계측 오버헤드로 **단일 실행 시간은 실행마다 편차가 크다**(같은 쿼리도 run마다 수 ms 출렁임). 따라서 각 문항의 튜닝 효과는 **실행계획 노드(스캔·조인·정렬)·버퍼·접근 행수의 변화**를 핵심 근거로 판단하였고(EXPLAIN ANALYZE, BUFFERS), 종합 요약 표의 실행시간은 각 문항 캡처의 **Execution Time(전→후)**을 그대로 옮긴 참고값이다. 이 이득은 데이터가 커질수록 확대된다.

0. 실습 환경 확인



▲ skala_db / ecom 스키마 접속(\conninfo), 테이블 목록·행 수 확인

제출물 1 · Q1~Q11 튜닝 전/후 비교

Q1. 지난 한 달간 실제 판매 총액 (paid/shipped/delivered) 인덱스 추가

요구사항 — 취소·환불·생성 상태 제외, 최근 1개월(now 기준) 매출 합계. 선택 컬럼: revenue 1개.

병목 — orders에 (상태 AND 기간) 필터가 있으나 이를 지원하는 인덱스가 없어 **Seq Scan on orders**(6,770행 스캔 → 5,339행 필터 제거).

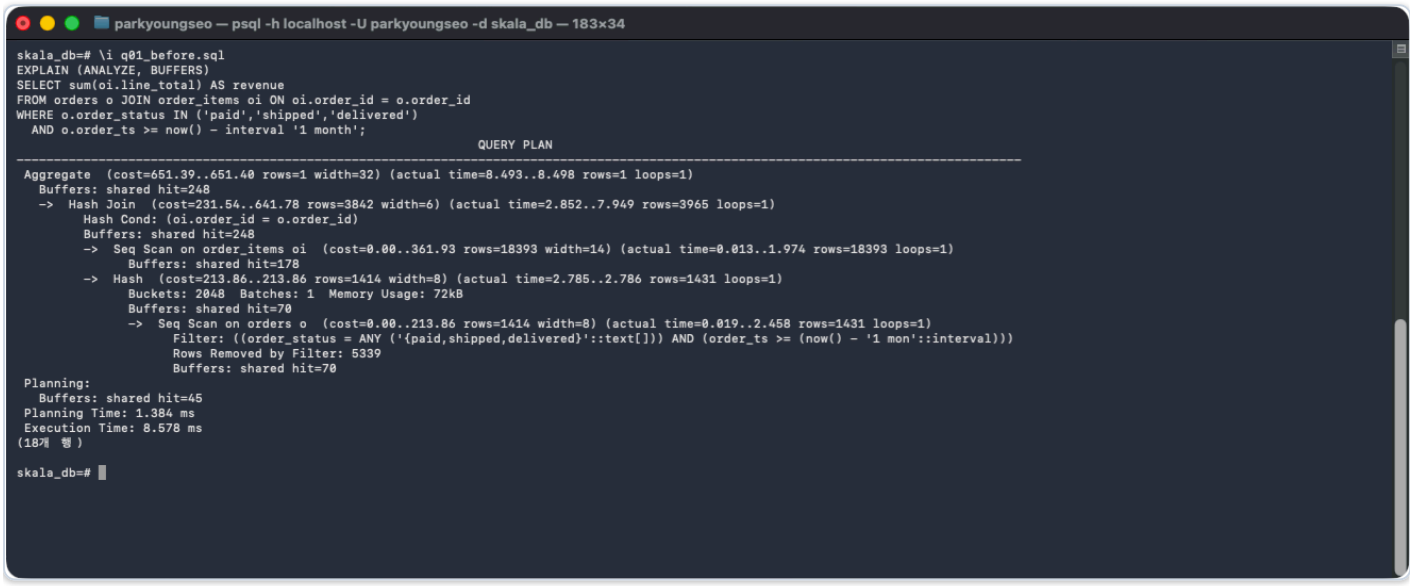
튜닝: 부분 인덱스 — 매출 상태만 담고 order_ts로 정렬

SQL — 튜닝 전 쿼리

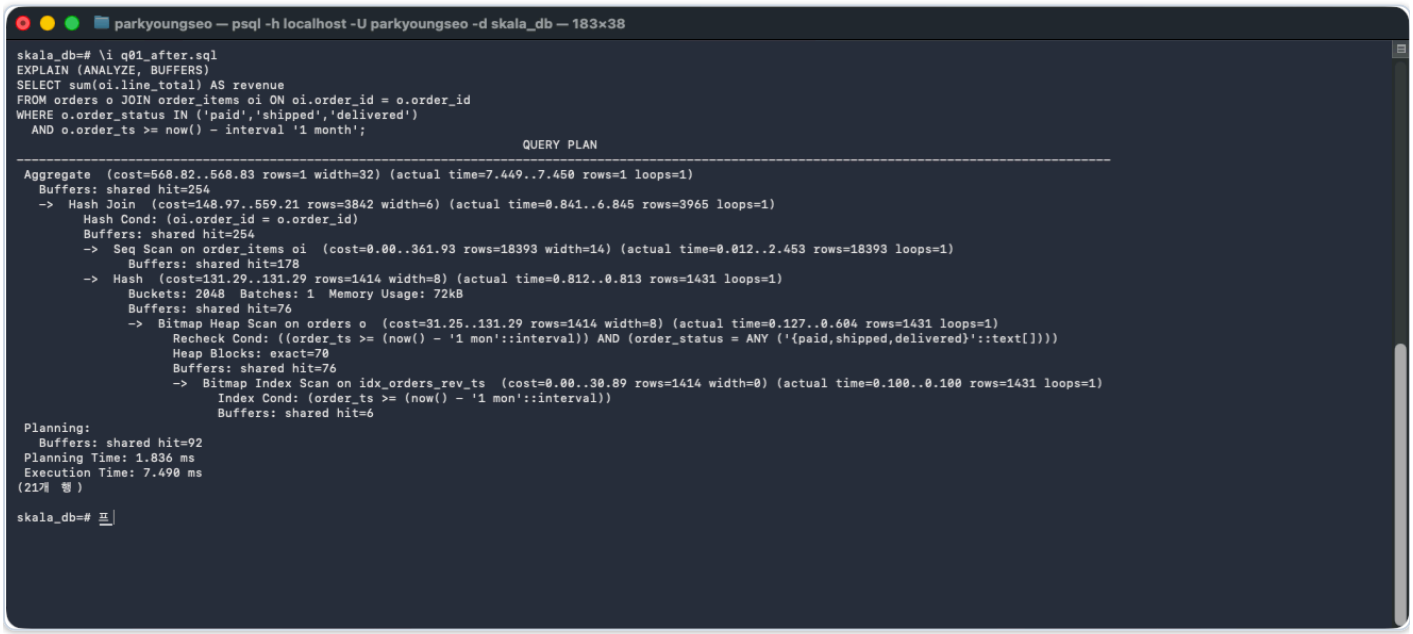
```sql
SELECT sum(oi.line_total) AS revenue                        -- 총매출 = 상세금액 전부 합산
FROM   orders o                                             -- 주문 헤더(상태/기간 필터 대상)
JOIN   order_items oi ON oi.order_id = o.order_id           -- 주문 1건에 딸린 상세 라인 연결
WHERE  o.order_status IN ('paid','shipped','delivered')     -- 매출 인정 상태만(취소/환불 제외)
  AND  o.order_ts >= now() - interval '1 month';            -- 최근 1개월로 기간 한정
```

SQL — 튜닝 후 (인덱스 DDL)

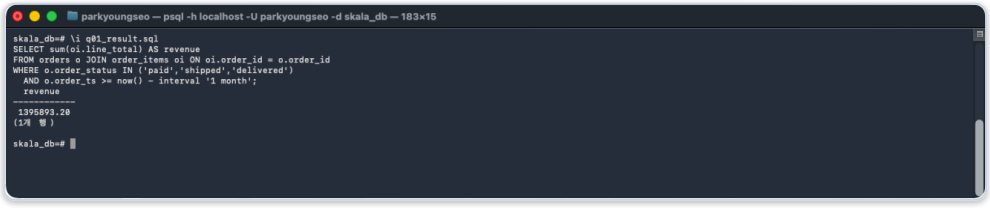
```sql
-- 부분 인덱스만 추가, 쿼리는 튜닝 전과 동일
CREATE INDEX idx_orders_rev_ts ON orders(order_ts)          -- order_ts를 정렬키로
  WHERE order_status IN ('paid','shipped','delivered');     -- 매출 상태만 인덱싱(부분 인덱스)
```



▲ 튜닝 전 — EXPLAIN (ANALYZE, BUFFERS) baseline 실행계획



▲ 튜닝 후 — 인덱스/재작성 적용 후 실행계획



▲ 실행 결과 (튜닝 전후 동일 — 정확성 보존)

> **개선** — orders 접근이 **Seq Scan(6,770행 전수) → Bitmap Index Scan (idx_orders_rev_ts, 1,431행)**. 필터 테이블을 인덱스로 직접 접근. 총 실행시간은 order_items 공유 스캔이 지배하여 유사하나, 데이터가 커질수록 인덱스 이득이 확대.

결과 — revenue = 1,395,893.20 (튜닝 전후 동일)

비즈니스 인사이트 — 지난 한 달 실결제 매출 약 **139만** — 취소·환불·생성을 제외한 '실제 확정 매출'만 집계하여 재무 마감·실적 지표로 바로 활용 가능.

Q2. 월별 주문 수 / 매출 / 주문당 평균금액(AOV) 쿼리 재작성

요구사항 — 월별 주문 건수·매출 합계·AOV(매출/주문수). 선택 컬럼: mon, orders, revenue, aov.

병목 — AOV용 **count(DISTINCT order_id)**가 GroupAggregate 입력을 (month, order_id)로 정렬 → **14,250행 Sort**(quicksort 941kB).

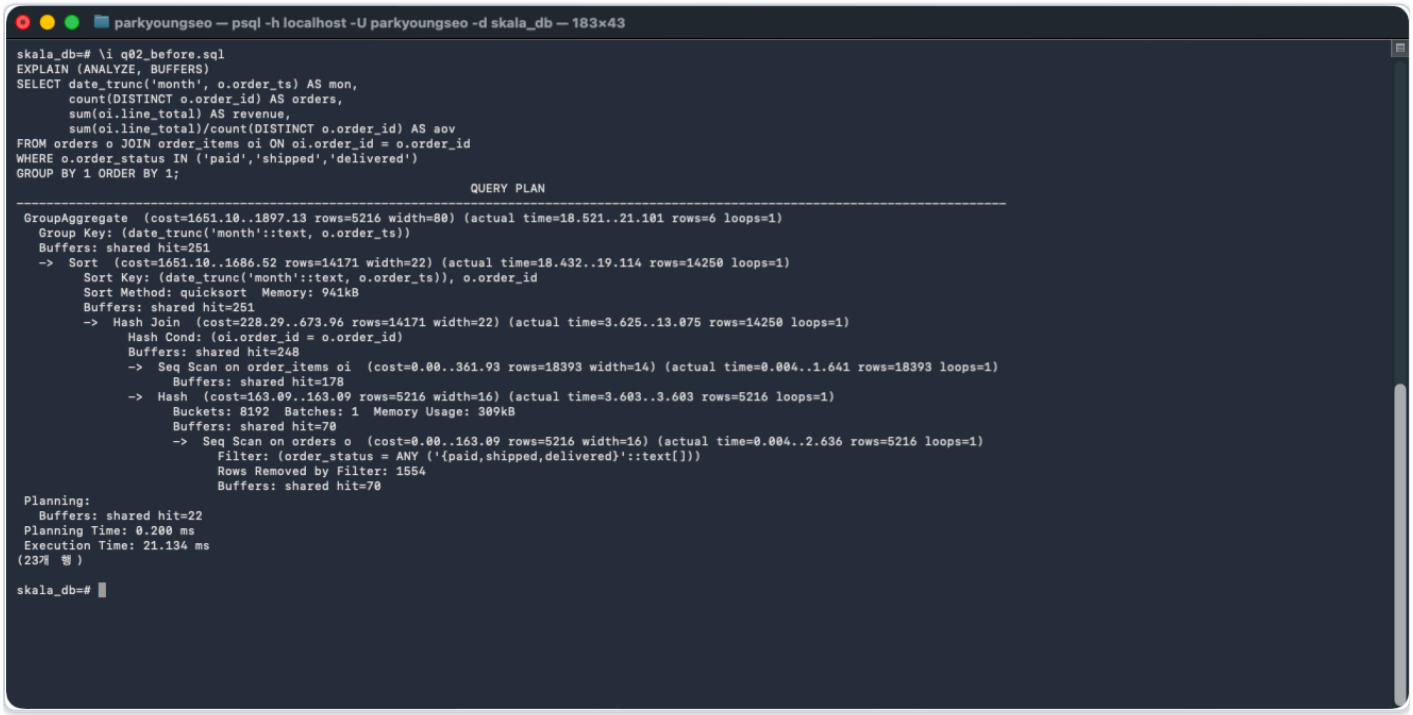
튜닝: 쿼리 재작성 — 주문 단위 선집계 후 월별 재집계 (count(DISTINCT) 제거)

SQL — 튜닝 전 쿼리

```
SELECT date_trunc('month', o.order_ts) AS mon,           -- 월 단위 절삭(월별 그룹키)
       count(DISTINCT o.order_id)        AS orders,      -- 주문수: 조인 중복 제거 위해 DISTINCT
       sum(oi.line_total)                AS revenue,     -- 월 매출 합계
       sum(oi.line_total)/count(DISTINCT o.order_id) AS aov  -- AOV = 매출 / 주문수
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
WHERE  o.order_status IN ('paid','shipped','delivered')  -- 매출 인정 상태만
GROUP BY 1 ORDER BY 1;                                   -- 월별로 그룹·정렬
```

SQL — 튜닝 후 (재작성 쿼리)

```
-- 주문 단위 선집계 후 월별 재집계 (count(DISTINCT) 제거)
SELECT date_trunc('month', order_ts) AS mon, count(*) AS orders,  -- 이미 주문단위 → count(*)
       sum(order_rev) AS revenue, round(sum(order_rev)/count(*),2) AS aov
FROM (
  SELECT o.order_id, o.order_ts, sum(oi.line_total) AS order_rev  -- 주문별 금액 선집계
  FROM   orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE  o.order_status IN ('paid','shipped','delivered')
  GROUP BY o.order_id, o.order_ts
) t
GROUP BY 1 ORDER BY 1;                                   -- 월별 재집계(HashAggregate)
```



▲ 튜닝 전 — EXPLAIN (ANALYZE, BUFFERS) baseline 실행계획

```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×51

skala_db=# \i q02_after.sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT date_trunc('month', order_ts) AS mon, count(*) AS orders,
       sum(order_rev) AS revenue, round(sum(order_rev)/count(*),2) AS aov
FROM (SELECT o.order_id, o.order_ts, sum(oi.line_total) AS order_rev
      FROM orders o JOIN order_items oi ON oi.order_id=o.order_id
      WHERE o.order_status IN ('paid','shipped','delivered')
      GROUP BY o.order_id, o.order_ts) t
GROUP BY 1 ORDER BY 1;
                                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=891.05..891.55 rows=200 width=80) (actual time=19.711..19.714 rows=6 loops=1)
   Sort Key: (date_trunc('month'::text, t.order_ts))
   Sort Method: quicksort  Memory: 25kB
   Buffers: shared hit=248
   ->  HashAggregate  (cost=878.91..883.41 rows=200 width=80) (actual time=19.701..19.707 rows=6 loops=1)
         Group Key: date_trunc('month'::text, t.order_ts)
         Batches: 1  Memory Usage: 40kB
         Buffers: shared hit=248
         ->  Subquery Scan on t  (cost=709.39..839.79 rows=5216 width=40) (actual time=15.661..18.741 rows=5216 loops=1)
               Buffers: shared hit=248
               ->  HashAggregate  (cost=709.39..774.59 rows=5216 width=48) (actual time=15.655..17.360 rows=5216 loops=1)
                     Group Key: o.order_id
                     Batches: 1  Memory Usage: 2257kB
                     Buffers: shared hit=248
                     ->  Hash Join  (cost=228.29..638.53 rows=14171 width=22) (actual time=2.837..10.172 rows=14250 loops=1)
                           Hash Cond: (oi.order_id = o.order_id)
                           Buffers: shared hit=248
                           ->  Seq Scan on order_items oi  (cost=0.00..361.93 rows=18393 width=14) (actual time=0.004..1.772 rows=18393 loops=1)
                                 Buffers: shared hit=178
                           ->  Hash  (cost=163.09..163.09 rows=5216 width=16) (actual time=2.824..2.824 rows=5216 loops=1)
                                 Buckets: 8192  Batches: 1  Memory Usage: 309kB
                                 Buffers: shared hit=70
                                 ->  Seq Scan on orders o  (cost=0.00..163.09 rows=5216 width=16) (actual time=0.004..2.066 rows=5216 loops=1)
                                       Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                                       Rows Removed by Filter: 1554
                                       Buffers: shared hit=70
 Planning:
   Buffers: shared hit=14
 Planning Time: 0.478 ms
 Execution Time: 19.760 ms
(30개 행)

skala_db=#
```

▲ 튜닝 후 — 인덱스/재작성 적용 후 실행계획

```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×23

skala_db=# \i q02_result.sql
SELECT date_trunc('month', order_ts)::date AS mon, count(*) AS orders,
       sum(order_rev) AS revenue, round(sum(order_rev)/count(*),2) AS aov
FROM (SELECT o.order_id, o.order_ts, sum(oi.line_total) AS order_rev
      FROM orders o JOIN order_items oi ON oi.order_id=o.order_id
      WHERE o.order_status IN ('paid','shipped','delivered')
      GROUP BY o.order_id, o.order_ts) t
GROUP BY 1 ORDER BY 1;
    mon     | orders |  revenue   |  aov
------------+--------+------------+--------
 2026-03-01 |    178 |  150853.58 | 847.49
 2026-04-01 |   1190 | 1077650.71 | 905.59
 2026-05-01 |   1253 | 1103544.71 | 880.72
 2026-06-01 |   1478 | 1397964.57 | 945.85
 2026-07-01 |   1115 | 1089484.21 | 977.12
 2026-08-01 |      2 |     700.10 | 350.05
(6개 행)

skala_db=#
```

▲ 실행 결과 (튜닝 전후 동일 — 정확성 보존)

개선 — **GroupAggregate + Sort(14,250행 정렬, 941kB) → 2단 HashAggregate**. count(DISTINCT)가 유발하던 대형 정렬 노드가 계획에서 사라짐.

결과 — 4~7월 월매출 108만~140만, AOV 880~977 (전후 동일)

비즈니스 인사이트 — 월 매출은 6월(약 **140만**)에 정점, **AOV가 880→977로 완만한 우상향** — 객단가 개선 추세(3·8월은 데이터 경계라 건수 적음).

Q3. 최근 90일 카테고리 Top10 (매출 기준) 계획 분석·크로스오버

요구사항 — 최근 90일, 카테고리별 매출 합계 상위 10. 선택 컬럼: category_id, category_name, revenue.

병목 — orders를 (상태 AND 90일) 필터로 Seq Scan 후 order_items·products·categories 4중 조인.

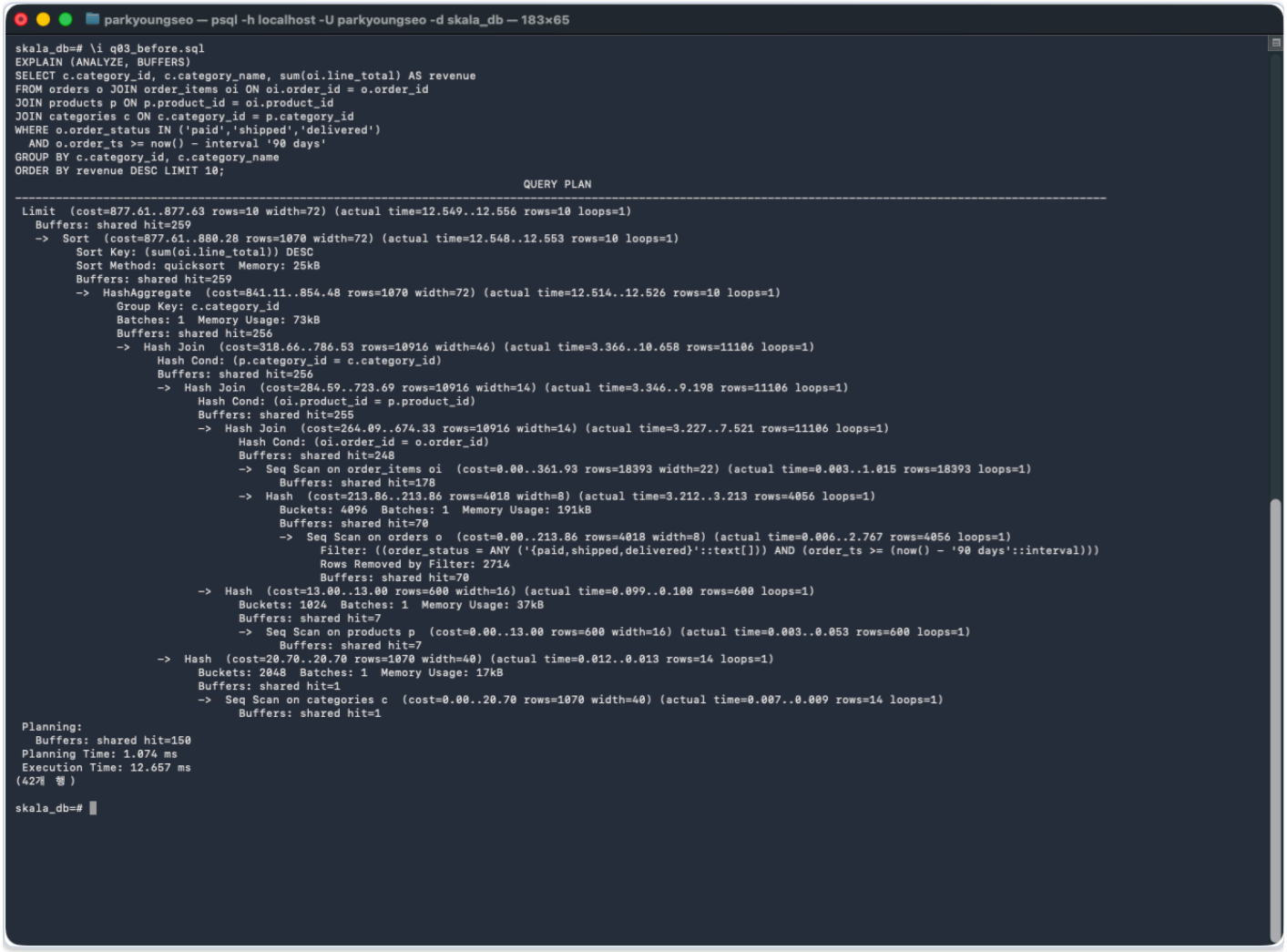
튜닝: 부분 인덱스(idx_orders_rev_ts) 활용 검토 — 단, 90일은 전체의 약 60%로 저선택도

SQL — 튜닝 전 쿼리

```
SELECT c.category_id, c.category_name, sum(oi.line_total) AS revenue  -- 카테고리별 매출
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id        -- 주문 → 상세
JOIN   products   p  ON p.product_id = oi.product_id     -- 상세 → 상품
JOIN   categories c  ON c.category_id = p.category_id    -- 상품 → 카테고리
WHERE  o.order_status IN ('paid','shipped','delivered')  -- 매출 인정 상태만
  AND  o.order_ts >= now() - interval '90 days'          -- 최근 90일
GROUP BY c.category_id, c.category_name
ORDER BY revenue DESC LIMIT 10;                          -- 매출 상위 10
```

튜닝 후 (계획 분석 — 쿼리 동일)

```
-- 90일 ≈ 60% 저선택도 → 옵티마이저가 Seq Scan 유지가 최적(강제 시 악화).
-- 부분 인덱스는 존재하나 이 조건에선 미채택. 쿼리 동일. (통계 최신화: ANALYZE)
--   검증: SET enable_seqscan=off → Bitmap Index Scan(idx_orders_rev_ts) 동작 확인
```



▲ 튜닝 전 — EXPLAIN (ANALYZE, BUFFERS) baseline 실행계획

parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×65

```
skala_db=# \i q03_after.sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.category_id, c.category_name, sum(oi.line_total) AS revenue
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
JOIN categories c ON c.category_id = p.category_id
WHERE o.order_status IN ('paid','shipped','delivered')
  AND o.order_ts >= now() - interval '90 days'
GROUP BY c.category_id, c.category_name
ORDER BY revenue DESC LIMIT 10;
                                                                          QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=877.61..877.63 rows=10 width=72) (actual time=13.838..13.843 rows=10 loops=1)
   Buffers: shared hit=256
   ->  Sort  (cost=877.61..880.28 rows=1070 width=72) (actual time=13.837..13.840 rows=10 loops=1)
         Sort Key: (sum(oi.line_total)) DESC
         Sort Method: quicksort  Memory: 25kB
         Buffers: shared hit=256
         ->  HashAggregate  (cost=841.11..854.48 rows=1070 width=72) (actual time=13.818..13.831 rows=10 loops=1)
               Group Key: c.category_id
               Batches: 1  Memory Usage: 73kB
               Buffers: shared hit=256
               ->  Hash Join  (cost=318.66..786.53 rows=10916 width=46) (actual time=3.506..11.775 rows=11106 loops=1)
                     Hash Cond: (p.category_id = c.category_id)
                     Buffers: shared hit=256
                     ->  Hash Join  (cost=284.59..723.69 rows=10916 width=14) (actual time=3.491..9.936 rows=11106 loops=1)
                           Hash Cond: (oi.product_id = p.product_id)
                           Buffers: shared hit=255
                           ->  Hash Join  (cost=264.09..674.33 rows=10916 width=14) (actual time=3.296..7.794 rows=11106 loops=1)
                                 Hash Cond: (oi.order_id = o.order_id)
                                 Buffers: shared hit=248
                                 ->  Seq Scan on order_items oi  (cost=0.00..361.93 rows=18393 width=22) (actual time=0.004..1.313 rows=18393 loops=1)
                                       Buffers: shared hit=178
                                 ->  Hash  (cost=213.86..213.86 rows=4018 width=8) (actual time=3.278..3.279 rows=4056 loops=1)
                                       Buckets: 4096  Batches: 1  Memory Usage: 191kB
                                       Buffers: shared hit=70
                                       ->  Seq Scan on orders o  (cost=0.00..213.86 rows=4018 width=8) (actual time=0.014..2.215 rows=4056 loops=1)
                                             Filter: ((order_status = ANY ('{paid,shipped,delivered}'::text[])) AND (order_ts >= (now() - '90 days'::interval)))
                                             Rows Removed by Filter: 2714
                                             Buffers: shared hit=70
                           ->  Hash  (cost=13.00..13.00 rows=600 width=16) (actual time=0.178..0.178 rows=600 loops=1)
                                 Buckets: 1024  Batches: 1  Memory Usage: 37kB
                                 Buffers: shared hit=7
                                 ->  Seq Scan on products p  (cost=0.00..13.00 rows=600 width=16) (actual time=0.003..0.121 rows=600 loops=1)
                                       Buffers: shared hit=7
                     ->  Hash  (cost=20.70..20.70 rows=1070 width=40) (actual time=0.011..0.011 rows=14 loops=1)
                           Buckets: 2048  Batches: 1  Memory Usage: 17kB
                           Buffers: shared hit=1
                           ->  Seq Scan on categories c  (cost=0.00..20.70 rows=1070 width=40) (actual time=0.007..0.008 rows=14 loops=1)
                                 Buffers: shared hit=1
 Planning:
   Buffers: shared hit=29
 Planning Time: 0.775 ms
 Execution Time: 13.884 ms
(42개 행)

skala_db=# ㅍ
```

▲ 튜닝 후 — 인덱스/재작성 적용 후 실행계획

parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×29

```
skala_db=# \i q03_result.sql
SELECT c.category_id, c.category_name, sum(oi.line_total) AS revenue
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
JOIN categories c ON c.category_id = p.category_id
WHERE o.order_status IN ('paid','shipped','delivered')
  AND o.order_ts >= now() - interval '90 days'
GROUP BY c.category_id, c.category_name
ORDER BY revenue DESC LIMIT 10;
 category_id | category_name |  revenue
-------------+---------------+------------
          10 | Women         | 1320413.25
          11 | Shoes         |  881934.06
          14 | Fitness       |  212921.92
           2 | Phones        |  204866.51
           4 | Audio         |  204647.32
           9 | Men           |  198222.47
          13 | Outdoor       |  193991.90
           3 | Laptops       |  192358.10
           6 | Appliances    |  190218.29
           7 | Cookware      |  184468.98
(10개 행)

skala_db=#
```

▲ 실행 결과 (튜닝 전후 동일 — 정확성 보존)

개선 — 이 규모(90일 ≈ 60%)에선 **Seq Scan 유지가 최적**(옵티마이저 판단 정당). ANALYZE로 categories 추정행 1,070→14 정확화. 강제 인덱스 시 오히려 악화 → 인덱스가 만능이 아님을 실증.

결과 — Women 132만 > Shoes 88만 > Fitness 21만 … (Top10, 전후 동일)

비즈니스 인사이트 — **Women·Shoes가 매출을 견인**(합계 약 220만), 이어 Fitness·Phones·Audio. 상위 2개 카테고리 집중도가 높아 재고·프로모션 우선순위 대상.

Q4. 제품별 누적 매출 RANK() Top20 계획 분석·크로스오버

요구사항 — 제품별 매출 합계에 RANK() 부여, 상위 20. 선택 컬럼: product_id, product_name, revenue, rnk.

병목 — 필터가 상태뿐이라 order_items 전량(18,393) 집계 불가피. RANK()는 정의상 전체 Sort 필요 → 계산 바운드.

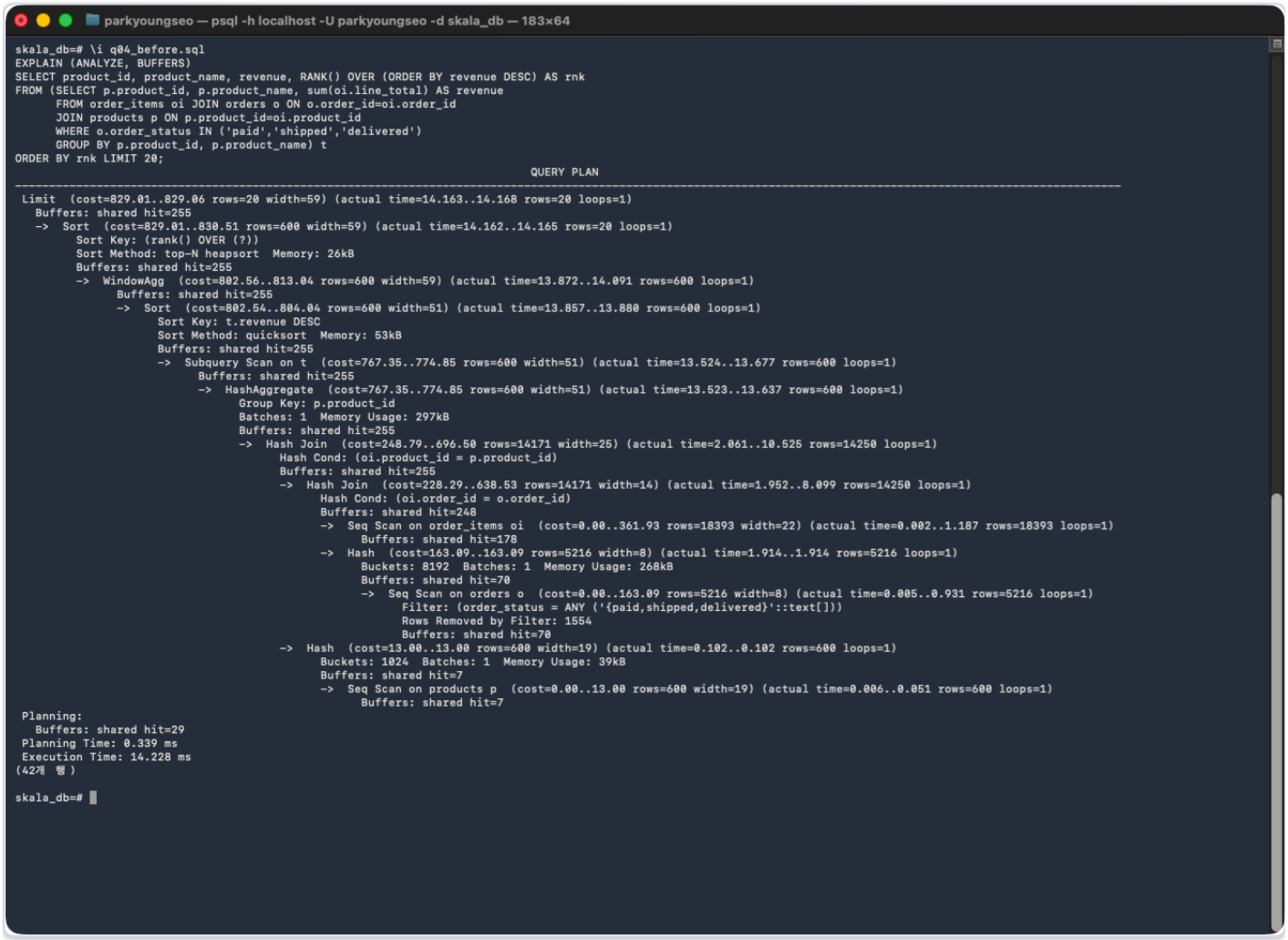
튜닝: 커버링 인덱스 검토 — 단, 전량 집계형이라 인덱스로는 스캔 축소 불가

SQL — 튜닝 전 쿼리

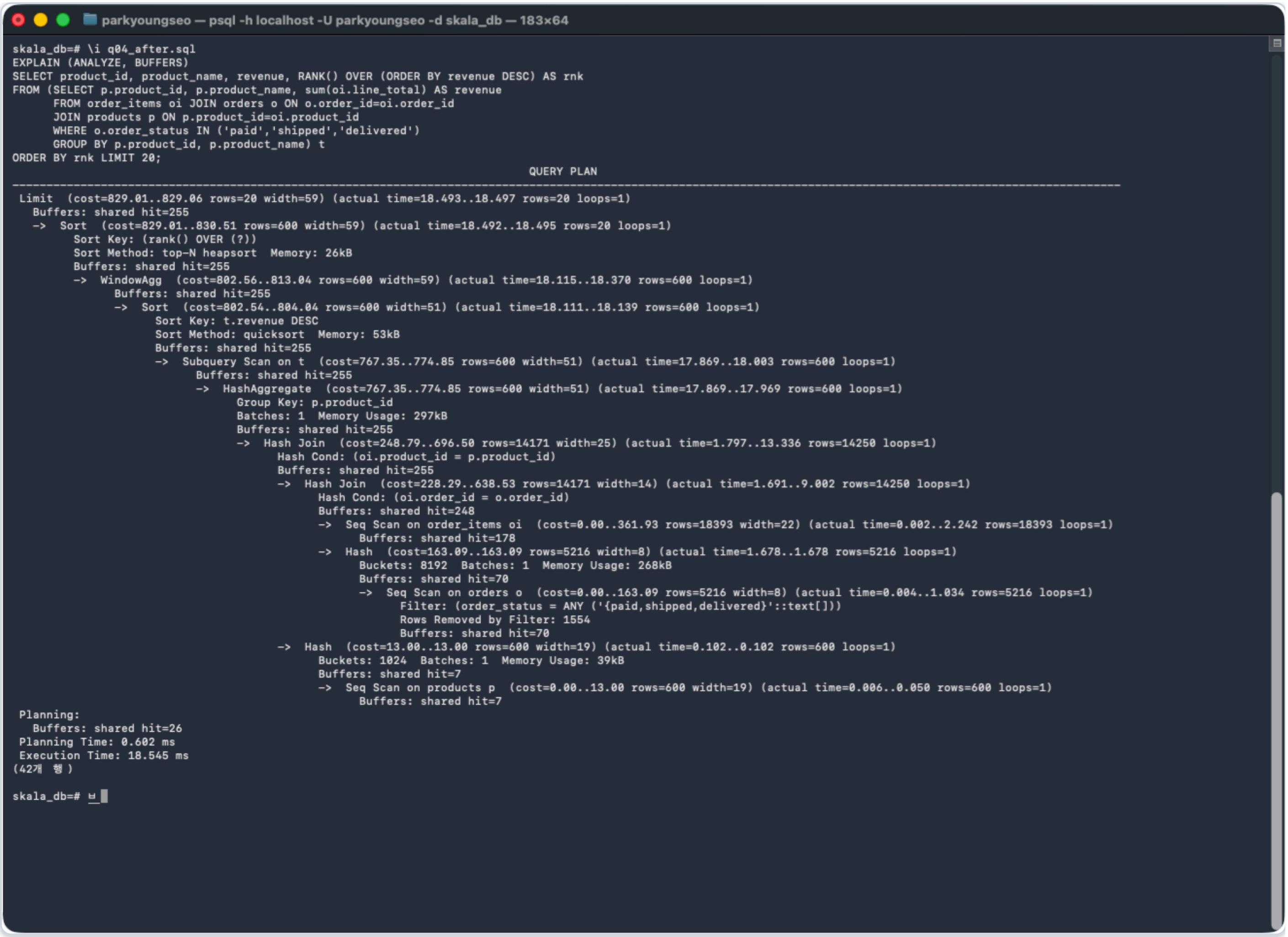
```
SELECT product_id, product_name, revenue,
       RANK() OVER (ORDER BY revenue DESC) AS rnk        -- 매출 내림차순 순위 부여
FROM (
  SELECT p.product_id, p.product_name, sum(oi.line_total) AS revenue  -- 제품별 매출 선집계
  FROM   order_items oi
  JOIN   orders   o ON o.order_id  = oi.order_id         -- 상태 필터용 주문 조인
  JOIN   products p ON p.product_id = oi.product_id      -- 상품명 조인
  WHERE  o.order_status IN ('paid','shipped','delivered')
  GROUP BY p.product_id, p.product_name
) t
ORDER BY rnk LIMIT 20;                                   -- 순위 상위 20
```

튜닝 후 (계획 분석 — 쿼리 동일)

```
-- 전량 집계 + RANK() 정렬은 불가피(계산 바운드) → 인덱스로 스캔 축소 불가.
-- 근본 개선책은 사전 집계(제출물 3의 Materialized View). 쿼리 동일.
```



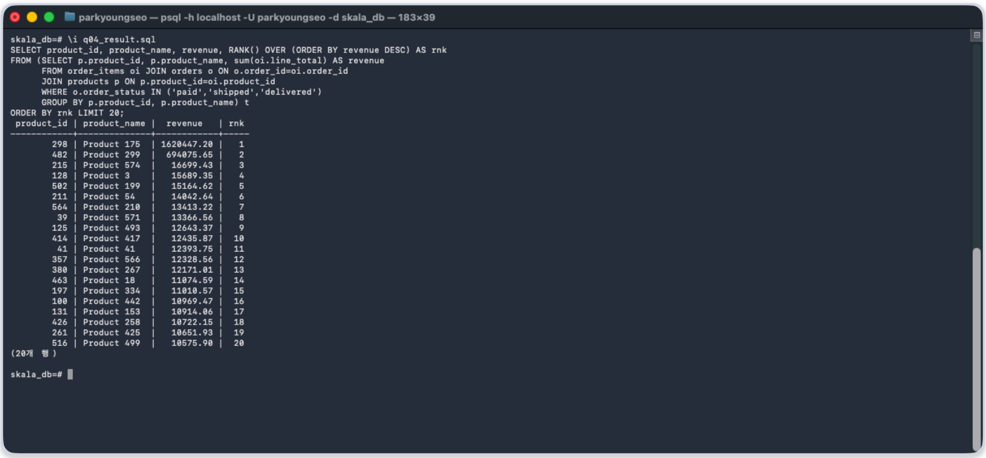
▲ 튜닝 전 — EXPLAIN (ANALYZE, BUFFERS) baseline 실행계획



```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×64
skala_db=# \i q04_after.sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT product_id, product_name, revenue, RANK() OVER (ORDER BY revenue DESC) AS rnk
FROM (SELECT p.product_id, p.product_name, sum(oi.line_total) AS revenue
      FROM order_items oi JOIN orders o ON o.order_id=oi.order_id
      JOIN products p ON p.product_id=oi.product_id
      WHERE o.order_status IN ('paid','shipped','delivered')
      GROUP BY p.product_id, p.product_name) t
ORDER BY rnk LIMIT 20;
                                                                        QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=829.01..829.06 rows=20 width=59) (actual time=18.493..18.497 rows=20 loops=1)
   Buffers: shared hit=255
   ->  Sort  (cost=829.01..830.51 rows=600 width=59) (actual time=18.492..18.495 rows=20 loops=1)
         Sort Key: (rank() OVER (?))
         Sort Method: top-N heapsort  Memory: 26kB
         Buffers: shared hit=255
         ->  WindowAgg  (cost=802.56..813.04 rows=600 width=59) (actual time=18.115..18.370 rows=600 loops=1)
               Buffers: shared hit=255
               ->  Sort  (cost=802.54..804.04 rows=600 width=51) (actual time=18.111..18.139 rows=600 loops=1)
                     Sort Key: t.revenue DESC
                     Sort Method: quicksort  Memory: 53kB
                     Buffers: shared hit=255
                     ->  Subquery Scan on t  (cost=767.35..774.85 rows=600 width=51) (actual time=17.869..18.003 rows=600 loops=1)
                           Buffers: shared hit=255
                           ->  HashAggregate  (cost=767.35..774.85 rows=600 width=51) (actual time=17.869..17.969 rows=600 loops=1)
                                 Group Key: p.product_id
                                 Batches: 1  Memory Usage: 297kB
                                 Buffers: shared hit=255
                                 ->  Hash Join  (cost=248.79..696.50 rows=14171 width=25) (actual time=1.797..13.336 rows=14250 loops=1)
                                       Hash Cond: (oi.product_id = p.product_id)
                                       Buffers: shared hit=255
                                       ->  Hash Join  (cost=228.29..638.53 rows=14171 width=14) (actual time=1.691..9.002 rows=14250 loops=1)
                                             Hash Cond: (oi.order_id = o.order_id)
                                             Buffers: shared hit=248
                                             ->  Seq Scan on order_items oi  (cost=0.00..361.93 rows=18393 width=22) (actual time=0.002..2.242 rows=18393 loops=1)
                                                   Buffers: shared hit=178
                                             ->  Hash  (cost=163.09..163.09 rows=5216 width=8) (actual time=1.678..1.678 rows=5216 loops=1)
                                                   Buckets: 8192  Batches: 1  Memory Usage: 268kB
                                                   Buffers: shared hit=70
                                                   ->  Seq Scan on orders o  (cost=0.00..163.09 rows=5216 width=8) (actual time=0.004..1.034 rows=5216 loops=1)
                                                         Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                                                         Rows Removed by Filter: 1554
                                                         Buffers: shared hit=70
                                       ->  Hash  (cost=13.00..13.00 rows=600 width=19) (actual time=0.102..0.102 rows=600 loops=1)
                                             Buckets: 1024  Batches: 1  Memory Usage: 39kB
                                             Buffers: shared hit=7
                                             ->  Seq Scan on products p  (cost=0.00..13.00 rows=600 width=19) (actual time=0.006..0.050 rows=600 loops=1)
                                                   Buffers: shared hit=7
 Planning:
   Buffers: shared hit=26
 Planning Time: 0.602 ms
 Execution Time: 18.545 ms
(42개 행)

skala_db=# ㅂ
```

▲ 튜닝 후 — 인덱스/재작성 적용 후 실행계획



```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×39
skala_db=# \i q04_result.sql
SELECT product_id, product_name, revenue, RANK() OVER (ORDER BY revenue DESC) AS rnk
FROM (SELECT p.product_id, p.product_name, sum(oi.line_total) AS revenue
      FROM order_items oi JOIN orders o ON o.order_id=oi.order_id
      JOIN products p ON p.product_id=oi.product_id
      WHERE o.order_status IN ('paid','shipped','delivered')
      GROUP BY p.product_id, p.product_name) t
ORDER BY rnk LIMIT 20;
 product_id | product_name |  revenue   | rnk
------------+--------------+------------+-----
        298 | Product 175  | 1620447.20 |   1
        482 | Product 299  |  694075.65 |   2
        215 | Product 574  |   16699.43 |   3
        128 | Product 3    |   15689.35 |   4
        502 | Product 199  |   15164.62 |   5
        211 | Product 54   |   14042.64 |   6
        564 | Product 210  |   13413.22 |   7
         39 | Product 571  |   13366.56 |   8
        125 | Product 493  |   12643.37 |   9
        414 | Product 417  |   12435.87 |  10
         41 | Product 41   |   12393.75 |  11
        357 | Product 566  |   12328.56 |  12
        380 | Product 267  |   12171.01 |  13
        463 | Product 18   |   11074.59 |  14
        197 | Product 334  |   11010.57 |  15
        100 | Product 442  |   10969.47 |  16
        131 | Product 153  |   10914.06 |  17
        426 | Product 258  |   10722.15 |  18
        261 | Product 425  |   10651.93 |  19
        516 | Product 499  |   10575.90 |  20
(20개 행)

skala_db=#
```

▲ 실행 결과 (튜닝 전후 동일 — 정확성 보존)

개선 — 필터가 상태뿐이라 order_items 전량 집계가 불가피하고 RANK() 정렬도 필수 → **전/후 계획 동일**(계산 바운드). 근본 개선책은 사전 집계(제출물 3의 MV).

결과 — 1위 Product 175(162만), 2위 Product 299(69만) … (전후 동일)

비즈니스 인사이트 — 상위 2개 제품(Product 175·299)이 162만·69만으로 압도적, 3위부터 급락 → 매출이 소수 히트상품에 편중(롱테일). 히트상품 품절 관리가 매출 방어의 핵심.

Q5. RFM 분석 — 최근성/빈도/금액 쿼리 재작성

요구사항 — 고객별 recency_days, frequency(주문수), monetary(매출). 금액 상위 20. 선택 컬럼 4개.

병목 — frequency = **count(DISTINCT order_id)** → GroupAggregate 입력 (customer_id, order_id) **14,250행 Sort**.

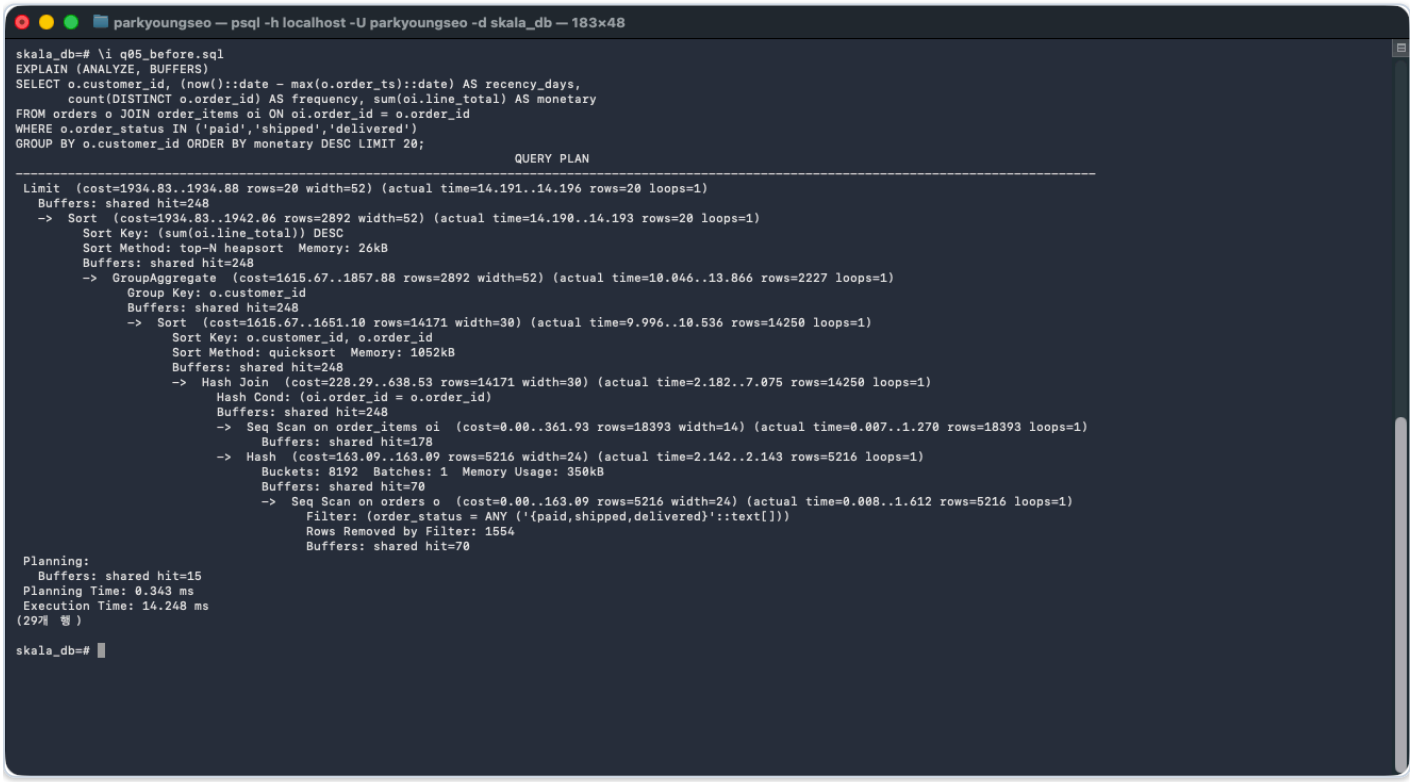
튜닝: 쿼리 재작성 — 주문 단위 선집계 후 고객별 재집계 (count(DISTINCT) 제거)

SQL — 튜닝 전 쿼리

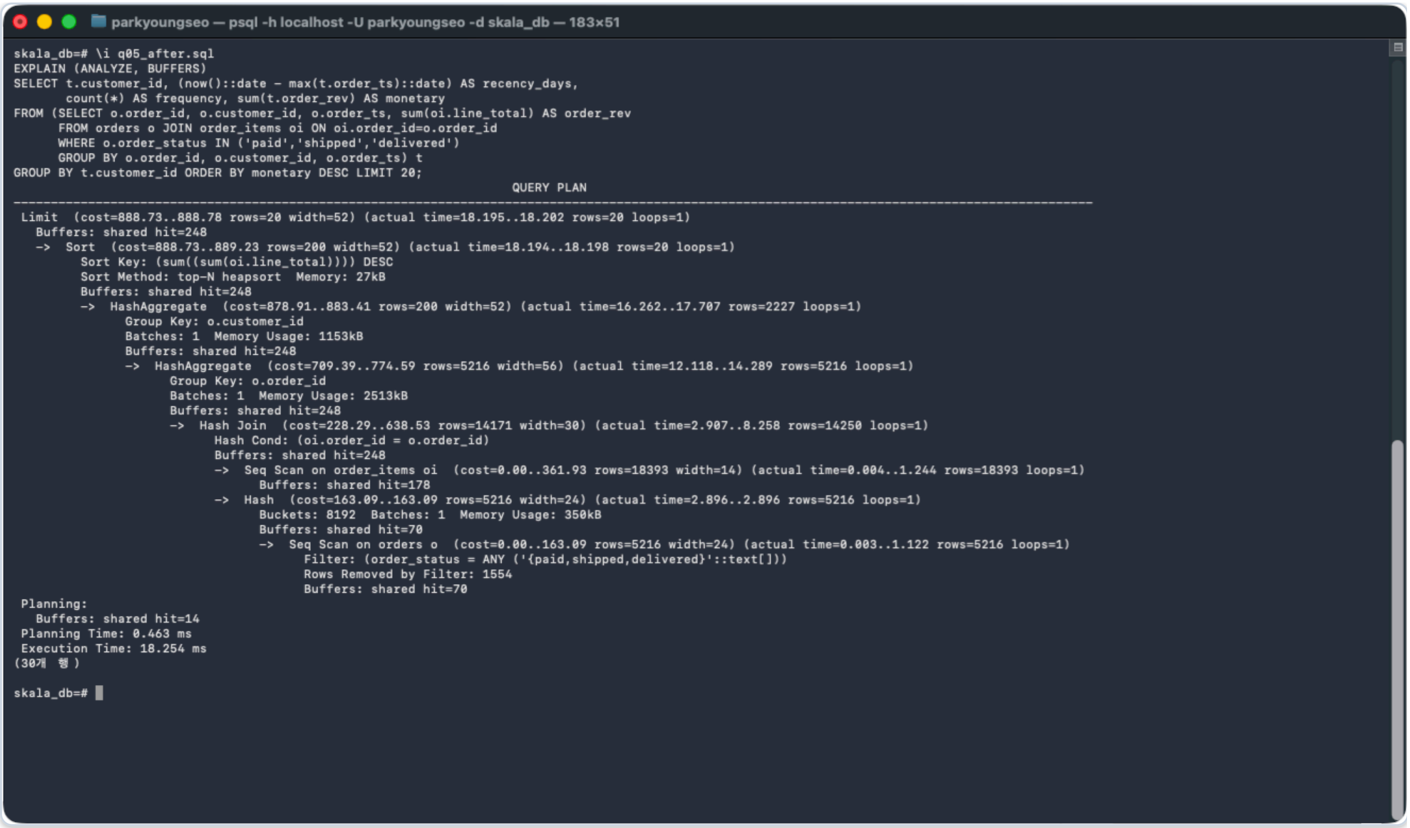
```
SELECT o.customer_id,
       (now()::date - max(o.order_ts)::date) AS recency_days,  -- R: 마지막 구매 후 경과일
       count(DISTINCT o.order_id)            AS frequency,     -- F: 주문 빈도(DISTINCT)
       sum(oi.line_total)                    AS monetary       -- M: 누적 구매액
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
WHERE  o.order_status IN ('paid','shipped','delivered')
GROUP BY o.customer_id
ORDER BY monetary DESC LIMIT 20;                          -- 금액 상위 20
```

SQL — 튜닝 후 (재작성 쿼리)

```
-- 주문 단위 선집계 후 고객별 재집계 (count(DISTINCT) 제거)
SELECT t.customer_id,
       (now()::date - max(t.order_ts)::date) AS recency_days,  -- R: 최근성
       count(*) AS frequency, sum(t.order_rev) AS monetary     -- F(=count*), M
FROM (
  SELECT o.order_id, o.customer_id, o.order_ts, sum(oi.line_total) AS order_rev  -- 주문별 선집계
  FROM   orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE  o.order_status IN ('paid','shipped','delivered')
  GROUP BY o.order_id, o.customer_id, o.order_ts
) t
GROUP BY t.customer_id ORDER BY monetary DESC LIMIT 20;  -- 고객별 재집계
```



▲ 튜닝 전 — EXPLAIN (ANALYZE, BUFFERS) baseline 실행계획

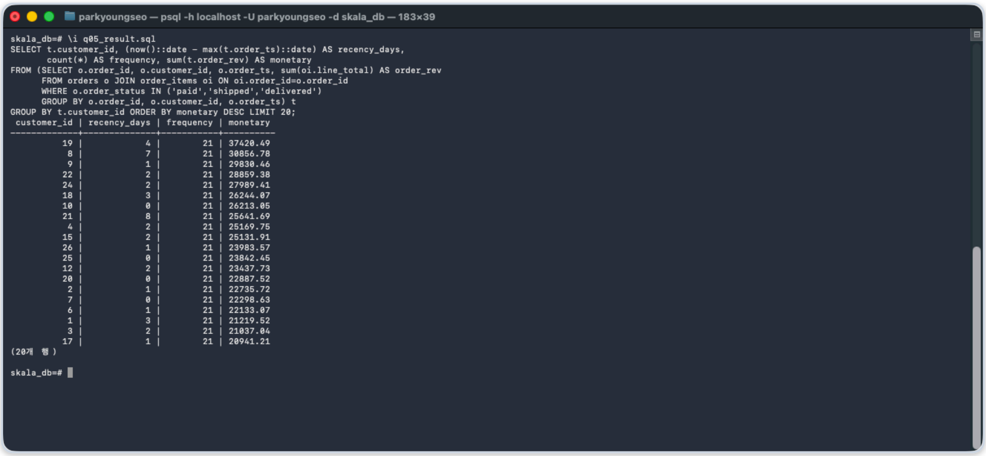


```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×51

skala_db=# \i q05_after.sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT t.customer_id, (now()::date - max(t.order_ts)::date) AS recency_days,
       count(*) AS frequency, sum(t.order_rev) AS monetary
FROM (SELECT o.order_id, o.customer_id, o.order_ts, sum(oi.line_total) AS order_rev
      FROM orders o JOIN order_items oi ON oi.order_id=o.order_id
      WHERE o.order_status IN ('paid','shipped','delivered')
      GROUP BY o.order_id, o.customer_id, o.order_ts) t
GROUP BY t.customer_id ORDER BY monetary DESC LIMIT 20;
                                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=888.73..888.78 rows=20 width=52) (actual time=18.195..18.202 rows=20 loops=1)
   Buffers: shared hit=248
   ->  Sort  (cost=888.73..889.23 rows=200 width=52) (actual time=18.194..18.198 rows=20 loops=1)
         Sort Key: (sum((sum(oi.line_total)))) DESC
         Sort Method: top-N heapsort  Memory: 27kB
         Buffers: shared hit=248
         ->  HashAggregate  (cost=878.91..883.41 rows=200 width=52) (actual time=16.262..17.707 rows=2227 loops=1)
               Group Key: o.customer_id
               Batches: 1  Memory Usage: 1153kB
               Buffers: shared hit=248
               ->  HashAggregate  (cost=709.39..774.59 rows=5216 width=56) (actual time=12.118..14.289 rows=5216 loops=1)
                     Group Key: o.order_id
                     Batches: 1  Memory Usage: 2513kB
                     Buffers: shared hit=248
                     ->  Hash Join  (cost=228.29..638.53 rows=14171 width=30) (actual time=2.907..8.258 rows=14250 loops=1)
                           Hash Cond: (oi.order_id = o.order_id)
                           Buffers: shared hit=248
                           ->  Seq Scan on order_items oi  (cost=0.00..361.93 rows=18393 width=14) (actual time=0.004..1.244 rows=18393 loops=1)
                                 Buffers: shared hit=178
                           ->  Hash  (cost=163.09..163.09 rows=5216 width=24) (actual time=2.896..2.896 rows=5216 loops=1)
                                 Buckets: 8192  Batches: 1  Memory Usage: 350kB
                                 Buffers: shared hit=70
                                 ->  Seq Scan on orders o  (cost=0.00..163.09 rows=5216 width=24) (actual time=0.003..1.122 rows=5216 loops=1)
                                       Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                                       Rows Removed by Filter: 1554
                                       Buffers: shared hit=70
 Planning:
   Buffers: shared hit=14
 Planning Time: 0.463 ms
 Execution Time: 18.254 ms
(30개 행)

skala_db=#
```

▲ 튜닝 후 — 인덱스/재작성 적용 후 실행계획



```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×39

skala_db=# \i q05_result.sql
SELECT t.customer_id, (now()::date - max(t.order_ts)::date) AS recency_days,
       count(*) AS frequency, sum(t.order_rev) AS monetary
FROM (SELECT o.order_id, o.customer_id, o.order_ts, sum(oi.line_total) AS order_rev
      FROM orders o JOIN order_items oi ON oi.order_id=o.order_id
      WHERE o.order_status IN ('paid','shipped','delivered')
      GROUP BY o.order_id, o.customer_id, o.order_ts) t
GROUP BY t.customer_id ORDER BY monetary DESC LIMIT 20;
 customer_id | recency_days | frequency | monetary
-------------+--------------+-----------+----------
          19 |            4 |        21 | 37420.49
           8 |            7 |        21 | 30856.78
           9 |            1 |        21 | 29830.46
          22 |            2 |        21 | 28859.38
          24 |            2 |        21 | 27989.41
          18 |            3 |        21 | 26244.07
          10 |            0 |        21 | 26213.05
          21 |            8 |        21 | 25641.69
           4 |            2 |        21 | 25169.75
          15 |            2 |        21 | 25131.91
          26 |            1 |        21 | 23983.57
          25 |            0 |        21 | 23842.45
          12 |            2 |        21 | 23437.73
          20 |            0 |        21 | 22887.52
           2 |            1 |        21 | 22735.72
           7 |            0 |        21 | 22298.63
           6 |            1 |        21 | 22133.07
           1 |            3 |        21 | 21219.52
           3 |            2 |        21 | 21037.04
          17 |            1 |        21 | 20941.21
(20개 행)

skala_db=#
```

▲ 실행 결과 (튜닝 전후 동일 — 정확성 보존)

개선 — **GroupAggregate + Sort(14,250행) → 2단 HashAggregate**. count(DISTINCT) 제거로 대형 정렬 노드 제거.

결과 — 헤비고객 19번(37,420) ~ 17번(20,941), frequency 21 균일 (전후 동일)

비즈니스 인사이트 — 상위 20 VIP가 **frequency 21로 균일하게 높고 monetary 2만~3.7만** → 헤비 고객층이 뚜렷. 이들 대상 리텐션·VIP 혜택 설계가 효과적.

Q6. 첫 구매 후 30일 내 재구매율 인덱스 추가

요구사항 — 고객 첫 결제완료 이후 30일 이내 재주문 비율(%). 선택 컬럼: cohort, repurchased, pct_30d.

병목 — LATERAL 프로브가 idx_orders_customer_ts로 Index Scan하나 status 필터를 **heap에서 재확인**(버퍼 hit 5,539).

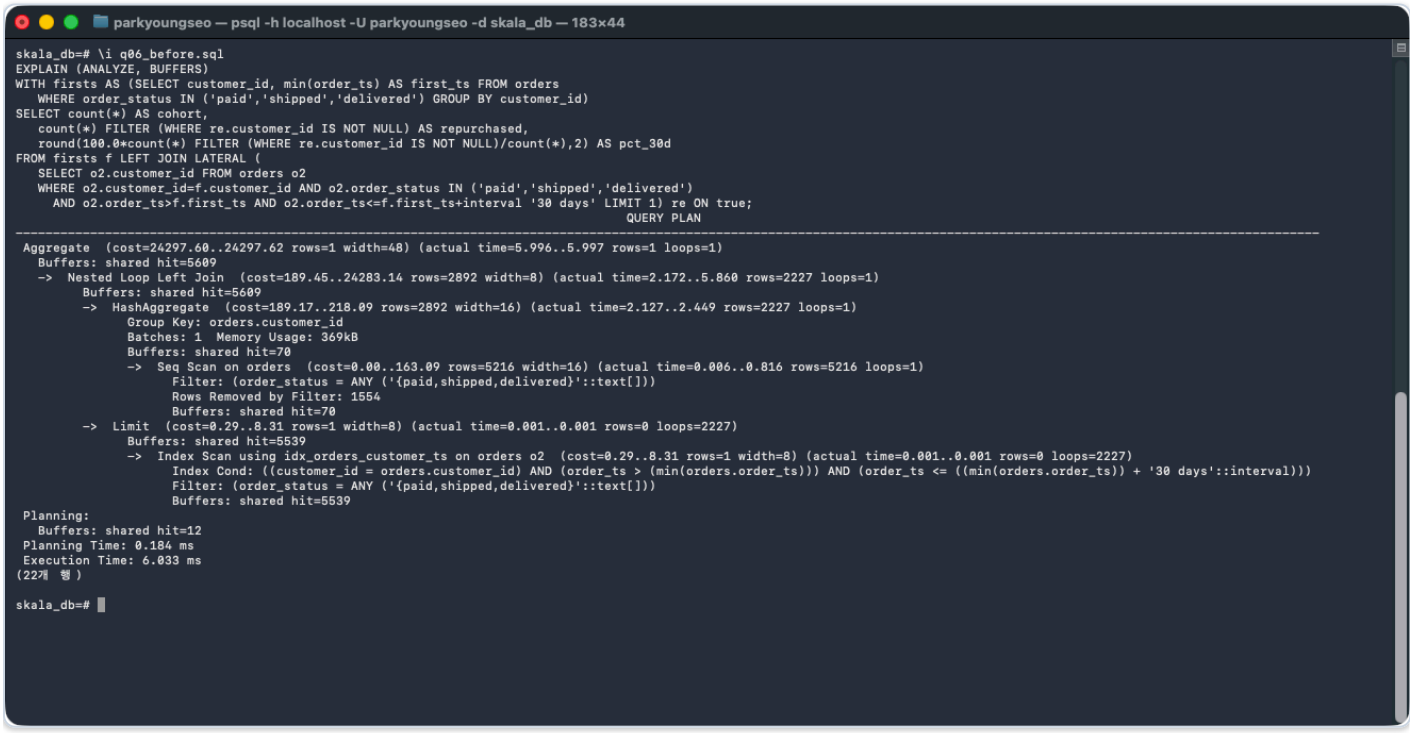
튜닝: 복합 부분 인덱스 — status 조건까지 인덱스에 포함

SQL — 튜닝 전 쿼리

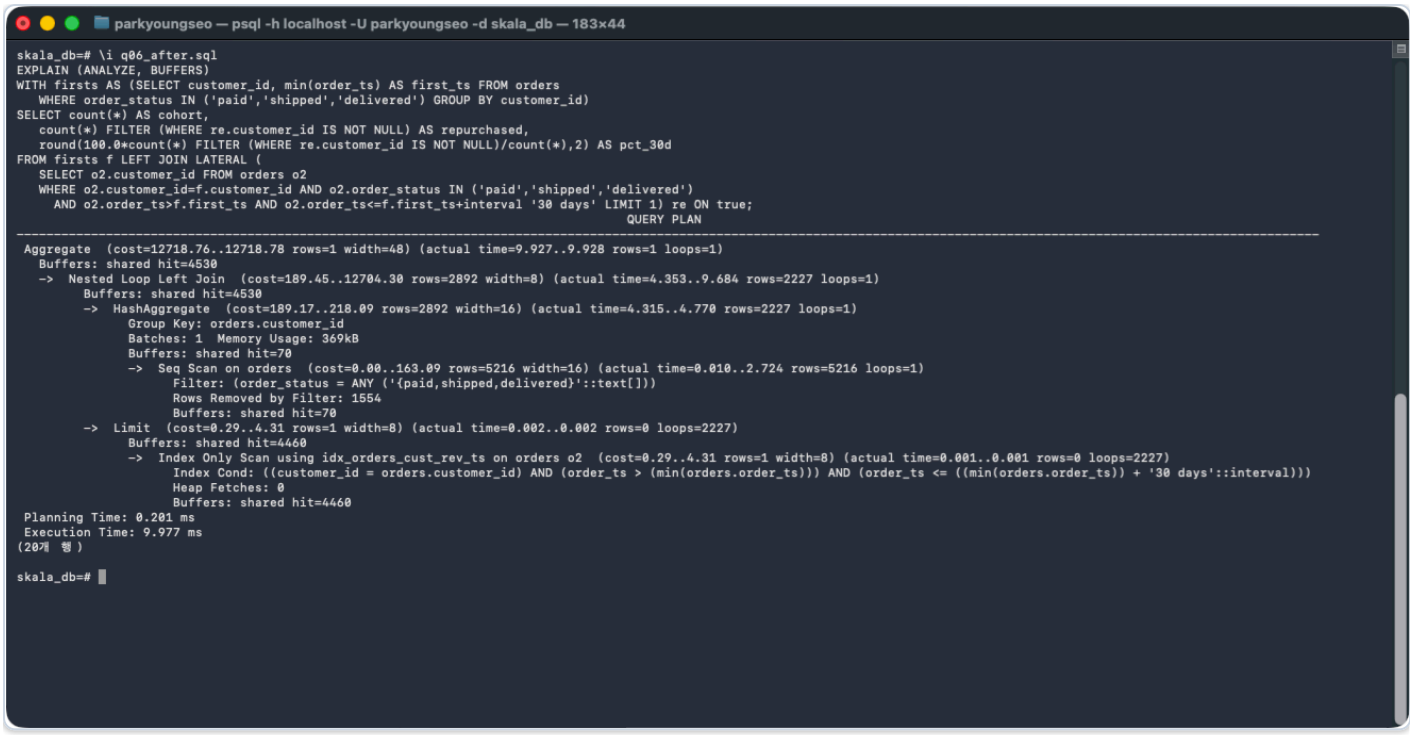
```
WITH firsts AS (                                        -- 고객별 첫 구매 시점 산출
  SELECT customer_id, min(order_ts) AS first_ts         -- 최초 주문시각 = 첫 구매일
  FROM orders WHERE order_status IN ('paid','shipped','delivered')
  GROUP BY customer_id
)
SELECT count(*) AS cohort,                              -- 첫 구매 고객 수(모수)
       count(*) FILTER (WHERE re.customer_id IS NOT NULL) AS repurchased,  -- 30일내 재구매자
       round(100.0*count(*) FILTER (WHERE re.customer_id IS NOT NULL)/count(*),2) AS pct_30d  -- 재구매율%
FROM   firsts f
LEFT JOIN LATERAL (                                     -- 각 첫구매행마다 재주문 탐색(프로브)
  SELECT o2.customer_id FROM orders o2
  WHERE o2.customer_id = f.customer_id                  -- 동일 고객
    AND o2.order_status IN ('paid','shipped','delivered')
    AND o2.order_ts >  f.first_ts                       -- 첫 구매 이후
    AND o2.order_ts <= f.first_ts + interval '30 days'  -- 30일 이내
  LIMIT 1                                               -- 1건이면 재구매 성립 → 조기 종료
) re ON true;
```

SQL — 튜닝 후 (인덱스 DDL)

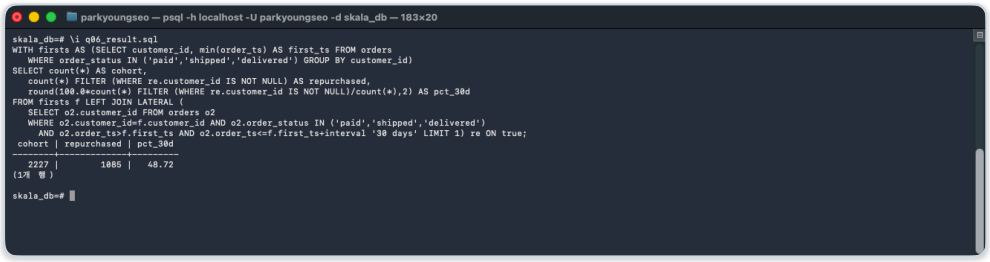
```
-- status 조건까지 담은 복합 부분 인덱스, 쿼리는 동일 (Index Only Scan 유도)
CREATE INDEX idx_orders_cust_rev_ts ON orders(customer_id, order_ts)  -- 프로브 복합키
  WHERE order_status IN ('paid','shipped','delivered');              -- 매출 상태만
```



▲ 튜닝 전 — EXPLAIN (ANALYZE, BUFFERS) baseline 실행계획



▲ 튜닝 후 — 인덱스/재작성 적용 후 실행계획



▲ 실행 결과 (튜닝 전후 동일 — 정확성 보존)

개선 — 재구매 탐색 프로브가 **Index Scan(heap 재확인, Buffers hit 5,539) → Index Only Scan (idx_orders_cust_rev_ts)**로 전환 → heap 접근 제거, 버퍼 대폭 감소.

결과 — cohort 2,227 / repurchased 1,085 / **pct_30d 48.72%** (전후 동일)

비즈니스 인사이트 — 첫 구매 후 30일 내 재구매율 48.72% → 신규의 절반이 한 달 안에 재구매. 초기 리텐션이 양호하므로 첫 구매 직후 30일 온보딩·리마케팅에 집중할 가치가 큼.

Q7. 재고 임계치 미만 상품 (곧 품절 위험) 인덱스 추가

요구사항 — qty_on_hand < reorder_point 상품, 부족량 큰 순. 선택 컬럼: product_id, name, qty, reorder_point, shortage.

병목 — inventory 전량(600) Seq Scan 후 두 컬럼 비교 필터(600→61).

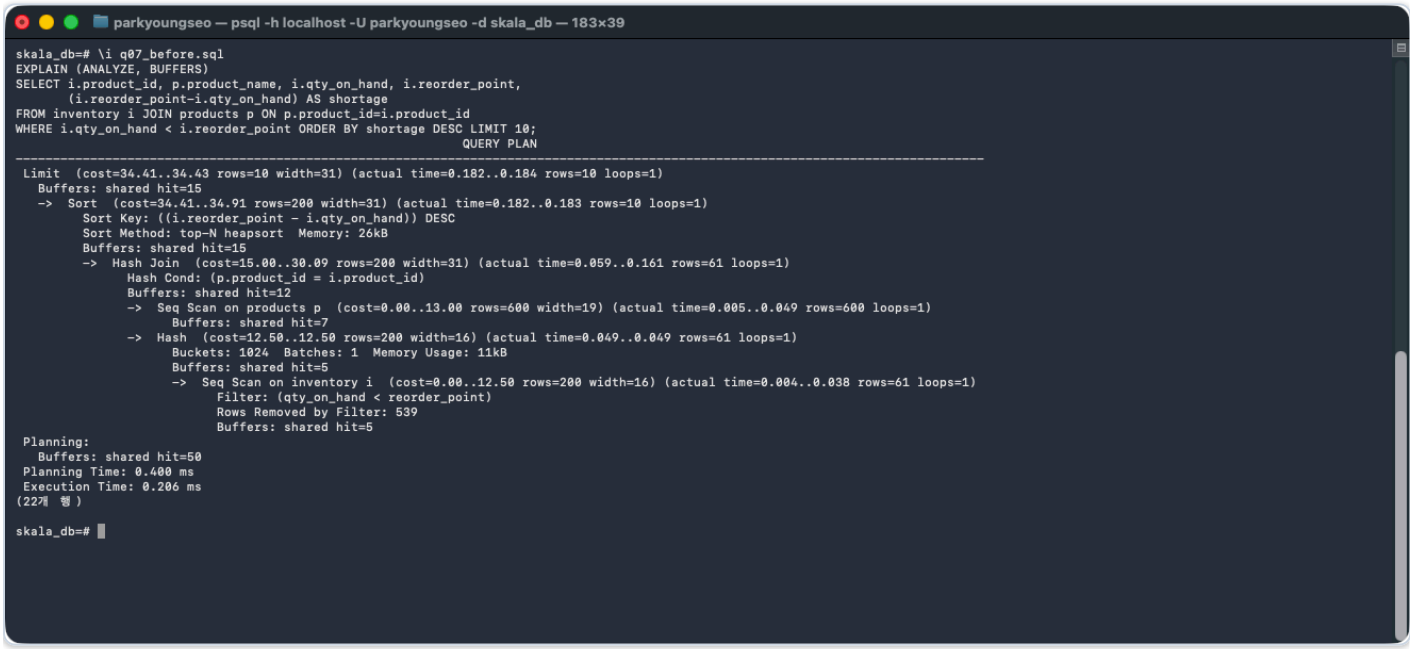
튜닝: 부분 인덱스 — 두 컬럼 비교 조건을 그대로 인덱스 조건으로 (품절 위험 상품만 인덱싱)

SQL — 튜닝 전 쿼리

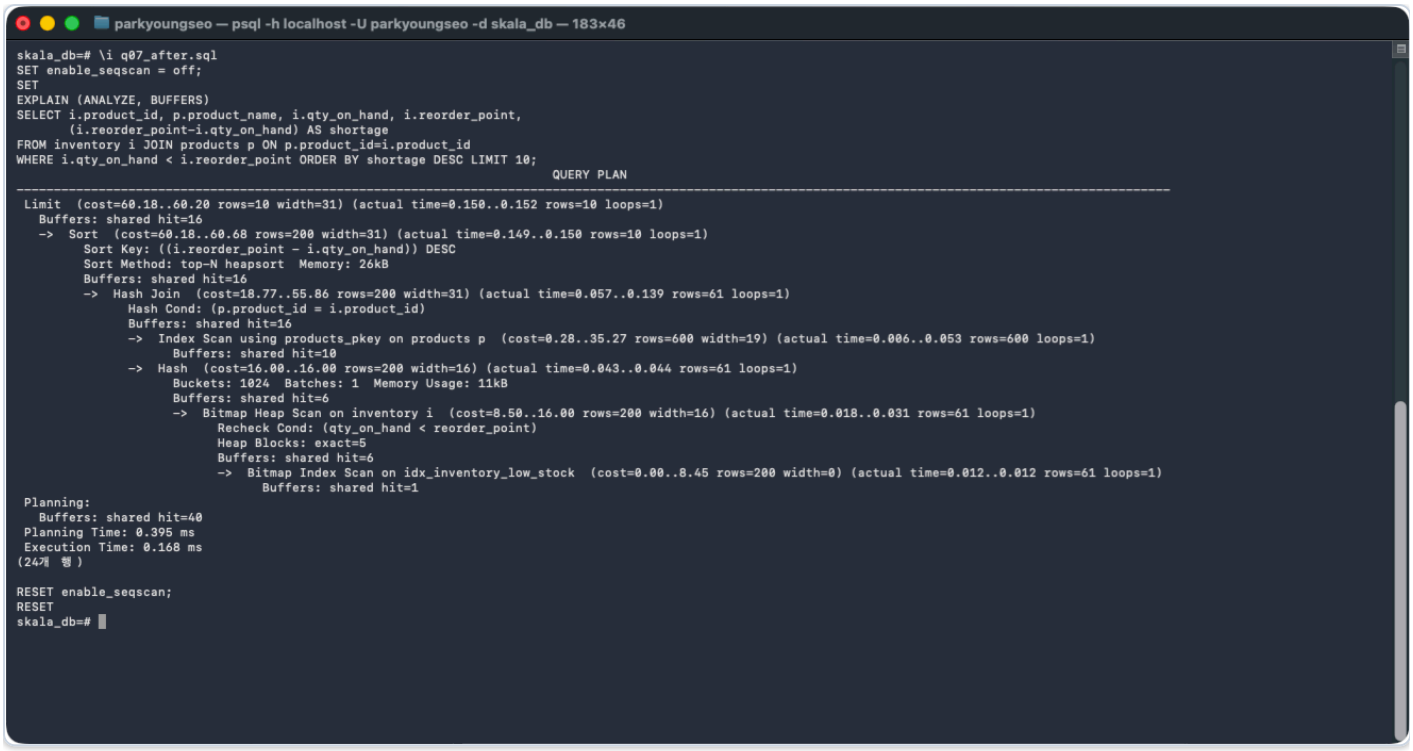
```
SELECT i.product_id, p.product_name, i.qty_on_hand, i.reorder_point,
       (i.reorder_point - i.qty_on_hand) AS shortage     -- 부족량 = 재주문점 - 현재고
FROM   inventory i
JOIN   products p ON p.product_id = i.product_id         -- 상품명 조인
WHERE  i.qty_on_hand < i.reorder_point                   -- 품절 위험(현재고 < 재주문점)
ORDER BY shortage DESC LIMIT 10;                         -- 부족량 큰 순(상위 10개 표시, 전체 위험 61건)
```

SQL — 튜닝 후 (인덱스 DDL)

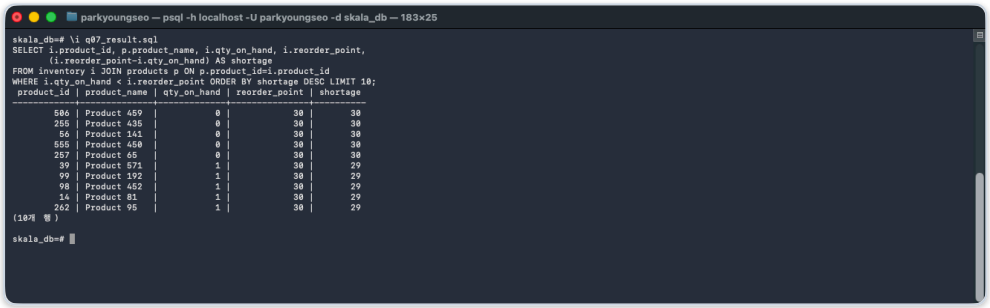
```
-- 두 컬럼 비교 조건을 그대로 담은 부분 인덱스, 쿼리는 동일
CREATE INDEX idx_inventory_low_stock ON inventory(product_id)  -- 위험 상품만 인덱싱
  WHERE qty_on_hand < reorder_point;
--   (600행 소형이라 옵티마이저는 Seq Scan 선택; 인덱스 활용 계획은
--    SET enable_seqscan=off로 검증 → Bitmap Index Scan 61행)
```



▲ 튜닝 전 — EXPLAIN (ANALYZE, BUFFERS) baseline 실행계획



▲ 튜닝 후 — 인덱스/재작성 적용 후 실행계획



▲ 실행 결과 (튜닝 전후 동일 — 정확성 보존)

개선 — **Seq Scan(600행 전수) → Bitmap Index Scan (idx_inventory_low_stock, 61행)**. 단, 600행 소형이라 두 방식의 절대시간 차이는 측정 노이즈 수준(sub-ms)이며, 옵티마이저는 비용 추정이 더 낮은 Seq Scan을 선택 — 접근 행수↓ 이득은 재고가 커질수록 뚜렷해진다.

결과 — 품절 위험 61건, 상위는 재고 0 / reorder_point 30 (shortage 30)

비즈니스 인사이트 — **품절 위험 61건, 그중 재고 0이 5건** → 즉시 발주 대상. 부족량(shortage) 내림차순으로 발주 우선순위를 바로 산출.

Q8. 효자상품 — 평점 4.5↑ & 리뷰 50↑ 인덱스 추가

요구사항 — HAVING avg(rating)>=4.5 AND count(*)>=50. 선택 컬럼: product_id, name, avg_rating, review_cnt.

병목 — reviews 전량(2,064) Seq Scan → products 조인 → HashAggregate + HAVING.

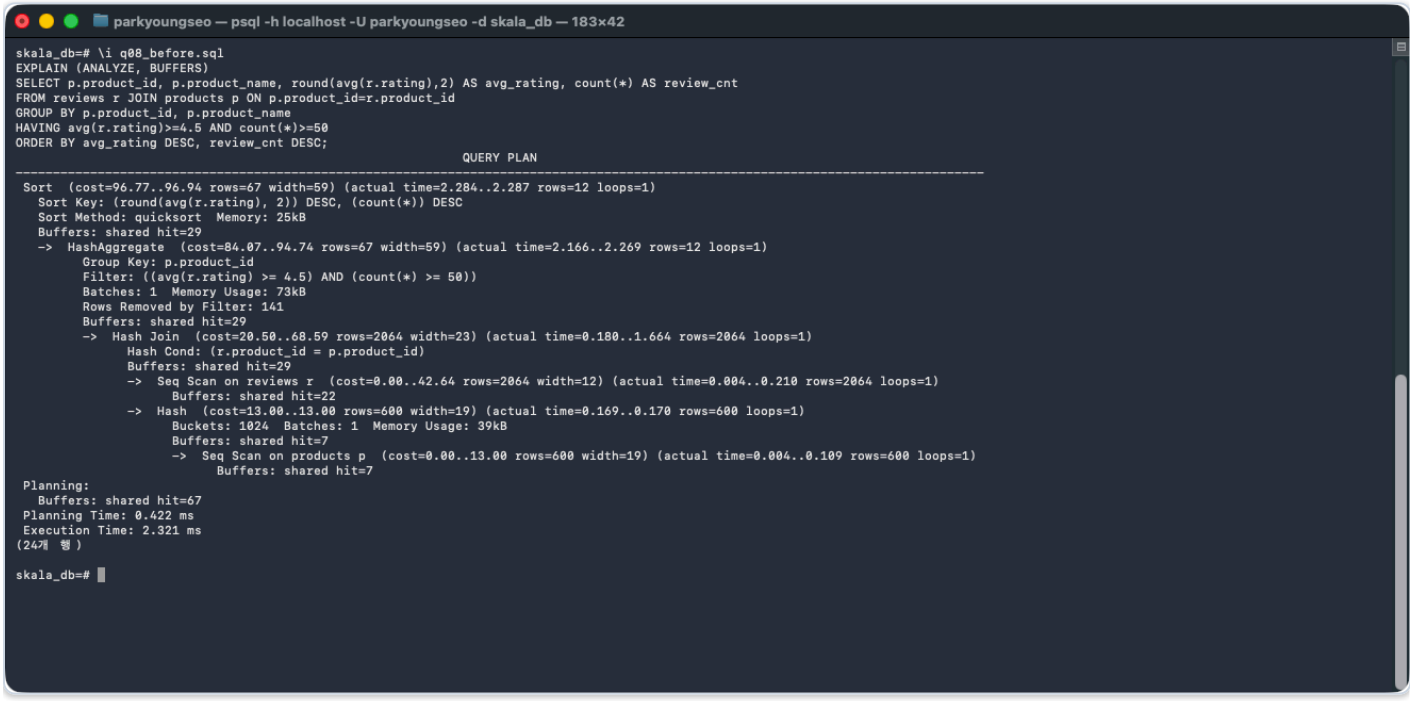
튜닝: 커버링 인덱스 — product_id로 그룹, rating은 INCLUDE (Index Only Scan)

SQL — 튜닝 전 쿼리

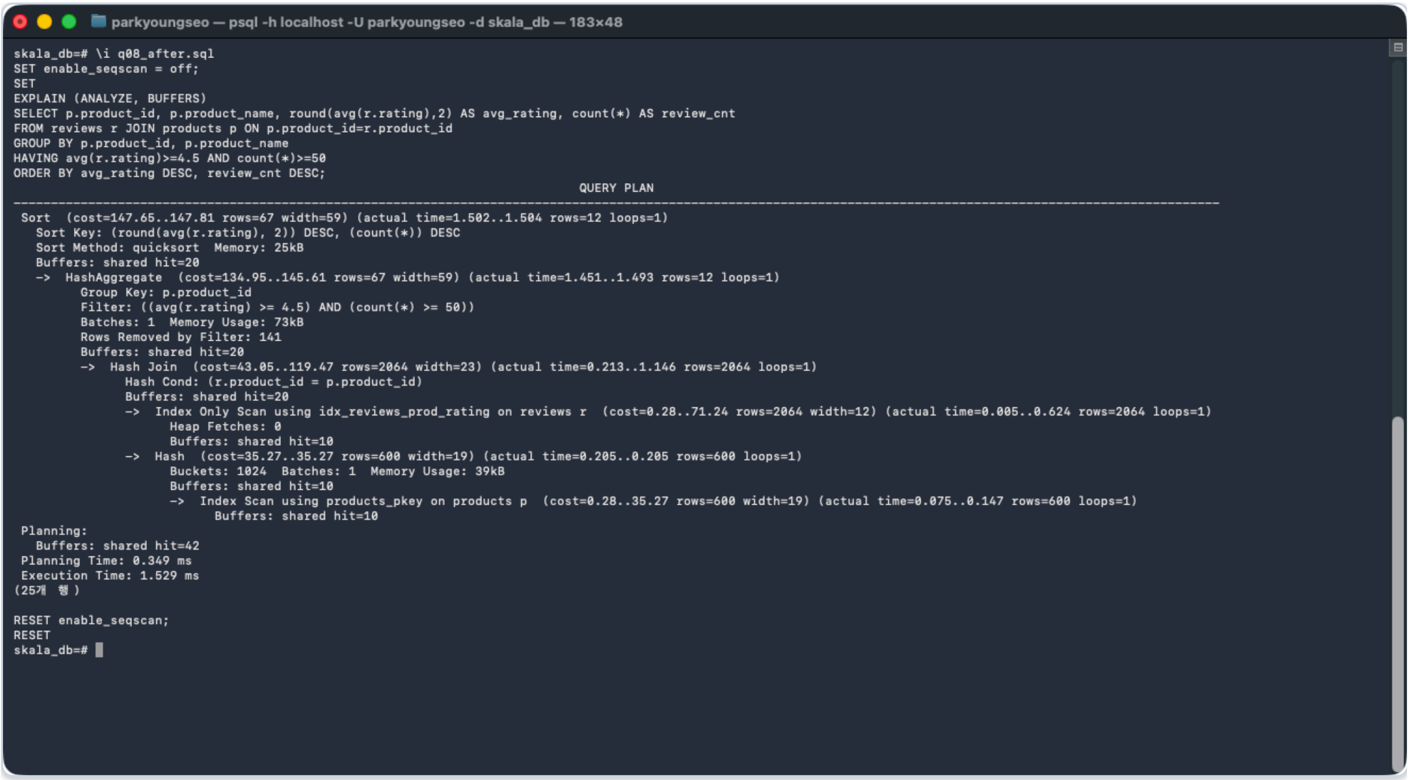
```
SELECT p.product_id, p.product_name,
       round(avg(r.rating),2) AS avg_rating, count(*) AS review_cnt  -- 평균평점·리뷰수
FROM   reviews r
JOIN   products p ON p.product_id = r.product_id
GROUP BY p.product_id, p.product_name
HAVING avg(r.rating) >= 4.5 AND count(*) >= 50           -- 효자상품: 고평점 & 표본 충분
ORDER BY avg_rating DESC, review_cnt DESC;
```

SQL — 튜닝 후 (인덱스 DDL)

```
-- product_id로 그룹, rating은 INCLUDE 한 커버링 인덱스, 쿼리는 동일
CREATE INDEX idx_reviews_prod_rating ON reviews(product_id) INCLUDE (rating);
--   (Index Only Scan 계획은 SET enable_seqscan=off로 검증 → Heap Fetches 0)
```



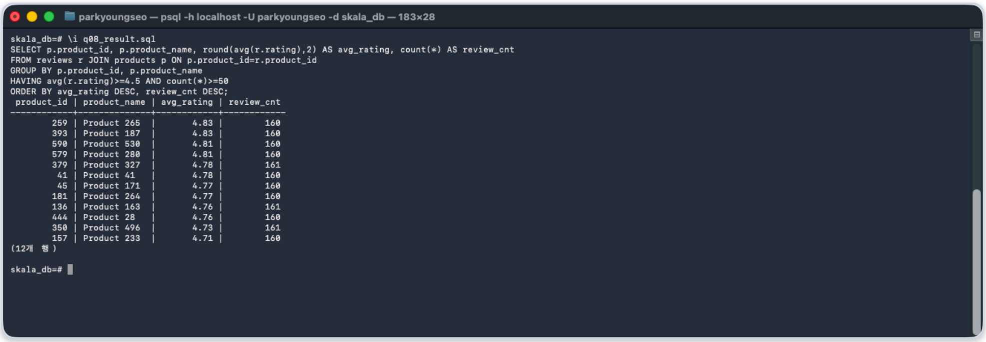
▲ 튜닝 전 — EXPLAIN (ANALYZE, BUFFERS) baseline 실행계획



```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×48
skala_db=# \i q08_after.sql
SET enable_seqscan = off;
SET
EXPLAIN (ANALYZE, BUFFERS)
SELECT p.product_id, p.product_name, round(avg(r.rating),2) AS avg_rating, count(*) AS review_cnt
FROM reviews r JOIN products p ON p.product_id=r.product_id
GROUP BY p.product_id, p.product_name
HAVING avg(r.rating)>=4.5 AND count(*)>=50
ORDER BY avg_rating DESC, review_cnt DESC;
                                                          QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=147.65..147.81 rows=67 width=59) (actual time=1.502..1.504 rows=12 loops=1)
   Sort Key: (round(avg(r.rating), 2)) DESC, (count(*)) DESC
   Sort Method: quicksort  Memory: 25kB
   Buffers: shared hit=20
   ->  HashAggregate  (cost=134.95..145.61 rows=67 width=59) (actual time=1.451..1.493 rows=12 loops=1)
         Group Key: p.product_id
         Filter: ((avg(r.rating) >= 4.5) AND (count(*) >= 50))
         Batches: 1  Memory Usage: 73kB
         Rows Removed by Filter: 141
         Buffers: shared hit=20
         ->  Hash Join  (cost=43.05..119.47 rows=2064 width=23) (actual time=0.213..1.146 rows=2064 loops=1)
               Hash Cond: (r.product_id = p.product_id)
               Buffers: shared hit=20
               ->  Index Only Scan using idx_reviews_prod_rating on reviews r  (cost=0.28..71.24 rows=2064 width=12) (actual time=0.005..0.624 rows=2064 loops=1)
                     Heap Fetches: 0
                     Buffers: shared hit=10
               ->  Hash  (cost=35.27..35.27 rows=600 width=19) (actual time=0.205..0.205 rows=600 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 39kB
                     Buffers: shared hit=10
                     ->  Index Scan using products_pkey on products p  (cost=0.28..35.27 rows=600 width=19) (actual time=0.075..0.147 rows=600 loops=1)
                           Buffers: shared hit=10
 Planning:
   Buffers: shared hit=42
 Planning Time: 0.349 ms
 Execution Time: 1.529 ms
(25개 행)

RESET enable_seqscan;
RESET
skala_db=#
```

▲ 튜닝 후 — 인덱스/재작성 적용 후 실행계획



```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×28
skala_db=# \i q08_result.sql
SELECT p.product_id, p.product_name, round(avg(r.rating),2) AS avg_rating, count(*) AS review_cnt
FROM reviews r JOIN products p ON p.product_id=r.product_id
GROUP BY p.product_id, p.product_name
HAVING avg(r.rating)>=4.5 AND count(*)>=50
ORDER BY avg_rating DESC, review_cnt DESC;
 product_id | product_name | avg_rating | review_cnt
------------+--------------+------------+------------
        259 | Product 265  |       4.83 |        160
        393 | Product 187  |       4.83 |        160
        590 | Product 530  |       4.81 |        160
        579 | Product 280  |       4.81 |        160
        379 | Product 327  |       4.78 |        161
         41 | Product 41   |       4.78 |        160
         45 | Product 171  |       4.77 |        160
        181 | Product 264  |       4.77 |        160
        136 | Product 163  |       4.76 |        161
        444 | Product 28   |       4.76 |        160
        350 | Product 496  |       4.73 |        161
        157 | Product 233  |       4.71 |        160
(12개 행)

skala_db=#
```

▲ 실행 결과 (튜닝 전후 동일 — 정확성 보존)

개선 — **Seq Scan → Index Only Scan (Heap Fetches: 0)** — (product_id, rating)만 읽어 heap I/O 제거. 2,064행 규모라 절대 시간은 유사하나 리뷰 폭증 대비 확장성 확보.

결과 — 효자상품 12개 (avg 4.71~4.83, 리뷰 160~161개, 전후 동일)

비즈니스 인사이트 — **효자상품 12개(평점 4.71~4.83, 리뷰 160+)** → 검증된 고평점·고리뷰 상품. 메인 노출·번들·리뷰 마케팅의 1순위 후보.

Q9. 쿠폰 사용 영향 — 쿠폰 有/無 AOV 비교 쿼리 재작성

요구사항 — coupon_code 유무별 주문수·AOV. 선택 컬럼: used_coupon, orders, aov.

병목 — 주문 단위 집계 후 바깥 GroupAggregate가 (used_coupon, order_id) 기준 **Sort(5,216행)** → 불필요 정렬.

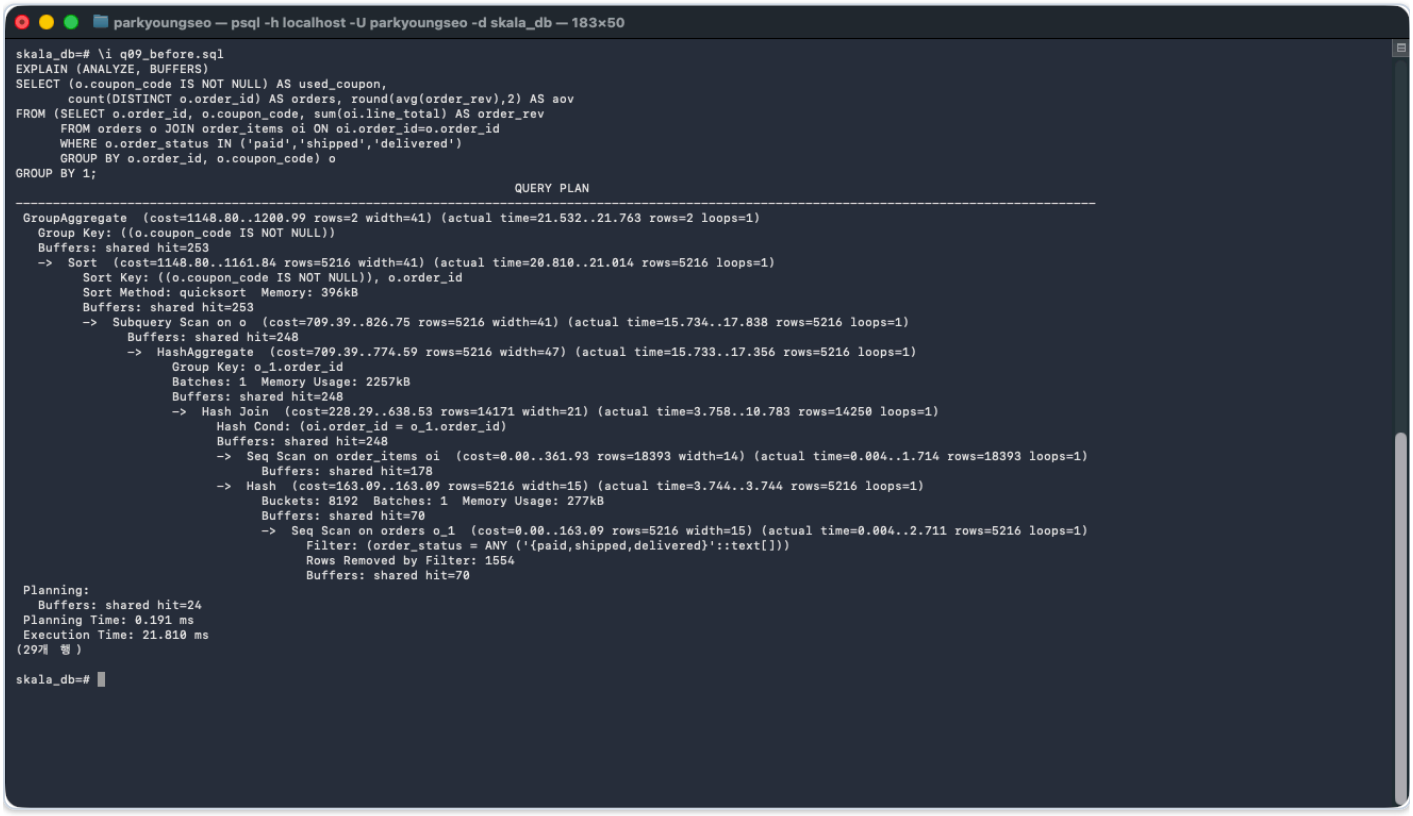
튜닝: 쿼리 재작성 — 안쪽에서 (order_id, used_coupon) 선집계 → 바깥 HashAggregate

SQL — 튜닝 전 쿼리

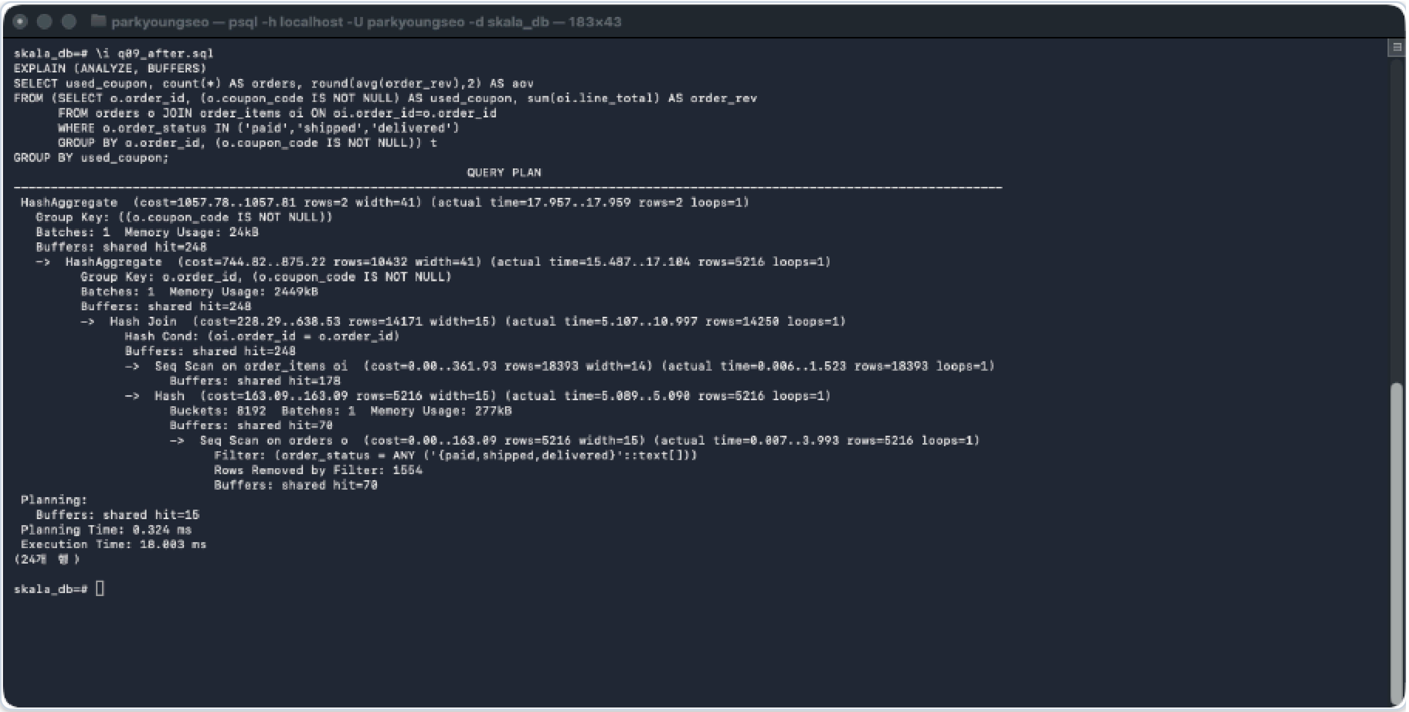
```
SELECT (o.coupon_code IS NOT NULL) AS used_coupon,       -- 쿠폰 사용 여부(T/F)
       count(DISTINCT o.order_id)       AS orders,       -- 주문수(DISTINCT)
       round(avg(order_rev),2)          AS aov           -- 평균 주문금액
FROM (
  SELECT o.order_id, o.coupon_code, sum(oi.line_total) AS order_rev  -- 주문 단위 선집계
  FROM   orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE  o.order_status IN ('paid','shipped','delivered')
  GROUP BY o.order_id, o.coupon_code
) o
GROUP BY 1;                                              -- 쿠폰 유무로 그룹
```

SQL — 튜닝 후 (재작성 쿼리)

```
-- 안쪽에서 (order_id, used_coupon) 선집계 → 바깥 HashAggregate (외부 Sort 제거)
SELECT used_coupon, count(*) AS orders, round(avg(order_rev),2) AS aov  -- 쿠폰유무별 집계
FROM (
  SELECT o.order_id, (o.coupon_code IS NOT NULL) AS used_coupon,
         sum(oi.line_total) AS order_rev                 -- 주문 단위 선집계
  FROM   orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE  o.order_status IN ('paid','shipped','delivered')
  GROUP BY o.order_id, (o.coupon_code IS NOT NULL)
) t
GROUP BY used_coupon;
```



▲ 튜닝 전 — EXPLAIN (ANALYZE, BUFFERS) baseline 실행계획



▲ 튜닝 후 — 인덱스/재작성 적용 후 실행계획

▲ 실행 결과 (튜닝 전후 동일 — 정확성 보존)

개선 — **GroupAggregate + Sort(5,216행) → HashAggregate**. 불필요한 외부 정렬 노드 제거.

결과 — 쿠폰無 3,953건 AOV 635.61 vs **쿠폰有 1,263건 AOV 1,827.10** (SAVE10이 고액주문에 집중)

비즈니스 인사이트 — **쿠폰 사용 주문 AOV 1,827 vs 미사용 635 — 약 2.9배**. 쿠폰이 고액 주문에 집중되어 업셀링·객단가 상승 효과를 시사(쿠폰=단순 할인이 아니라 큰 장바구니를 유도).

Q10. 상위 1% 고객의 최근 60일 매출 인덱스 추가

요구사항 — 최근 60일 매출 상위 1%(99퍼센타일↑) 고객. 선택 컬럼: customer_id, rev.

병목 — 최근 60일 매출을 고객별 집계. orders를 (상태 AND 60일) 필터로 **Seq Scan**(3,894행 제거).

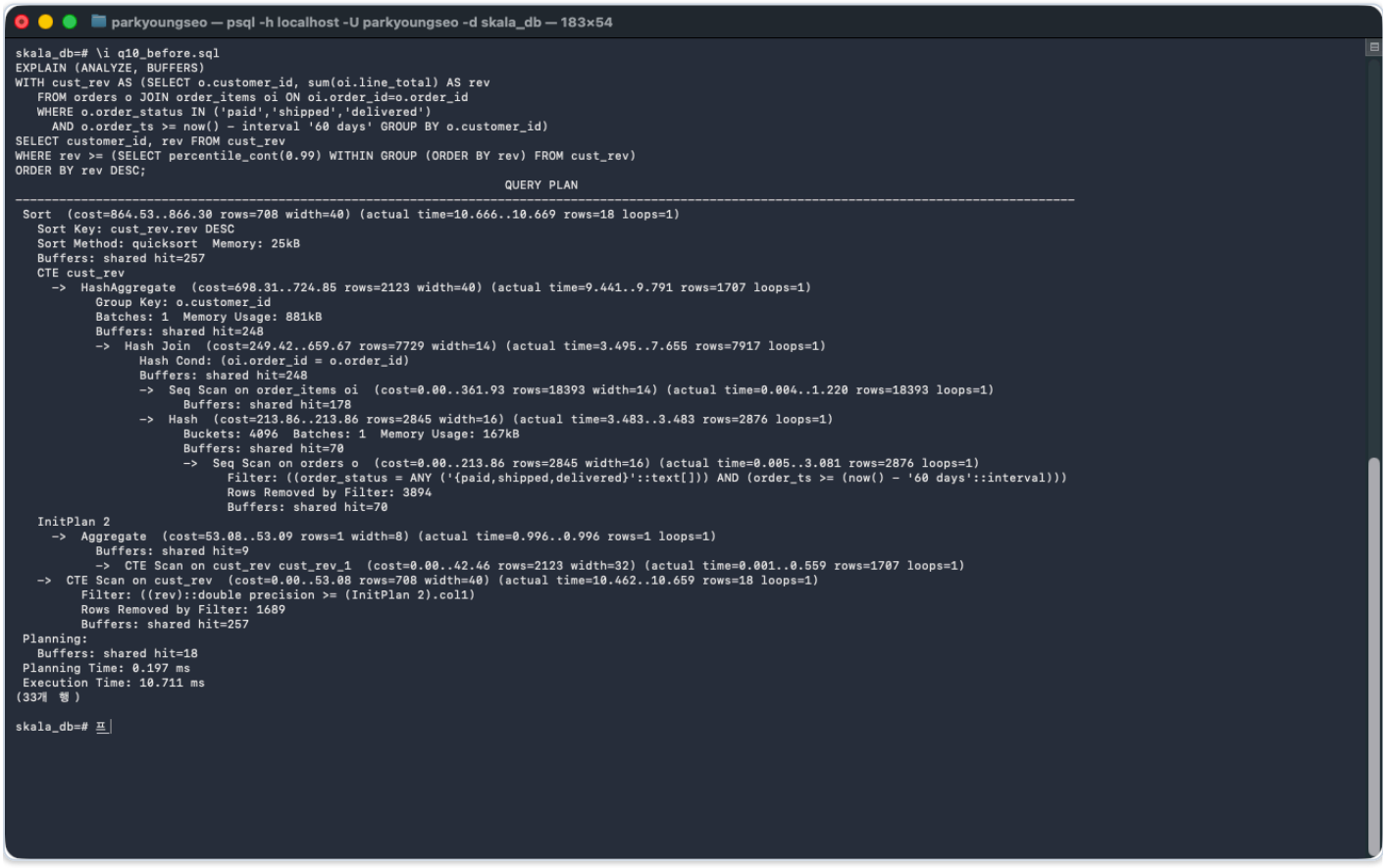
튜닝: 부분 인덱스(idx_orders_rev_ts) 활용 — 60일=약 42%로 Q3보다 선택적

SQL — 튜닝 전 쿼리

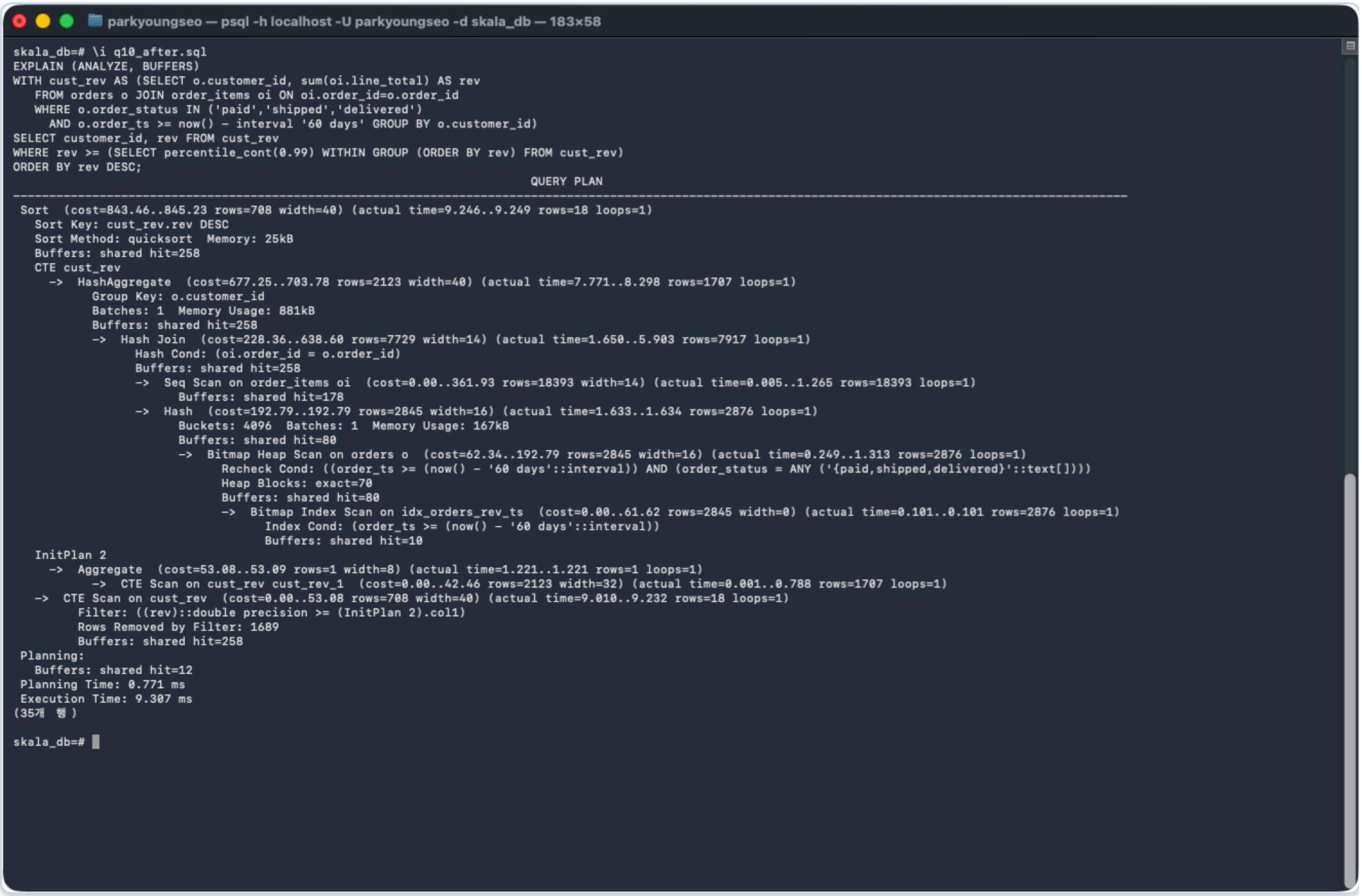
```
WITH cust_rev AS (                                        -- 최근 60일 고객별 매출
  SELECT o.customer_id, sum(oi.line_total) AS rev
  FROM   orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE  o.order_status IN ('paid','shipped','delivered')
    AND  o.order_ts >= now() - interval '60 days'         -- 최근 60일
  GROUP BY o.customer_id
)
SELECT customer_id, rev FROM cust_rev
WHERE rev >= (SELECT percentile_cont(0.99) WITHIN GROUP (ORDER BY rev) FROM cust_rev)  -- 상위1% 임계값 이상
ORDER BY rev DESC;
```

SQL — 튜닝 후 (인덱스 DDL)

```
-- Q1과 동일 부분 인덱스 재사용(60일 ≈ 42%로 선택적), 쿼리는 동일
CREATE INDEX idx_orders_rev_ts ON orders(order_ts)       -- order_ts 정렬키
  WHERE order_status IN ('paid','shipped','delivered');  -- 매출 상태만(부분 인덱스)
```



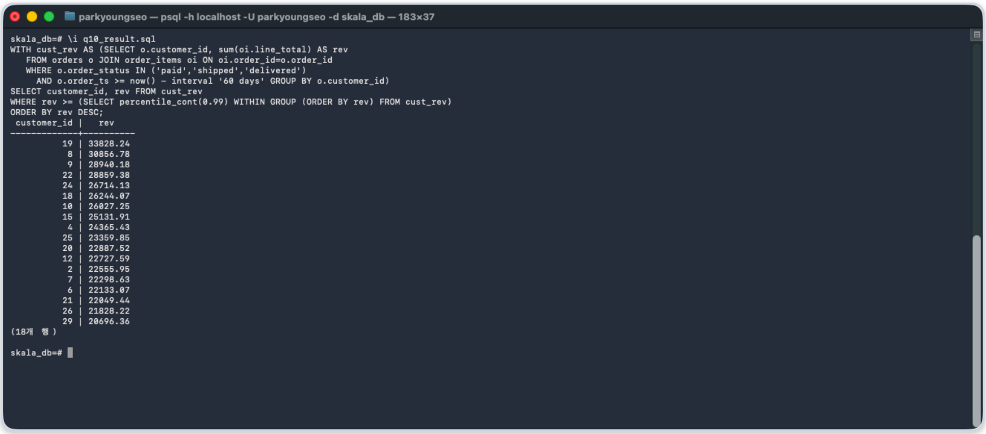
▲ 튜닝 전 — EXPLAIN (ANALYZE, BUFFERS) baseline 실행계획



```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×58
skala_db=# \i q10_after.sql
EXPLAIN (ANALYZE, BUFFERS)
WITH cust_rev AS (SELECT o.customer_id, sum(oi.line_total) AS rev
   FROM orders o JOIN order_items oi ON oi.order_id=o.order_id
   WHERE o.order_status IN ('paid','shipped','delivered')
     AND o.order_ts >= now() - interval '60 days' GROUP BY o.customer_id)
SELECT customer_id, rev FROM cust_rev
WHERE rev >= (SELECT percentile_cont(0.99) WITHIN GROUP (ORDER BY rev) FROM cust_rev)
ORDER BY rev DESC;
                                                                     QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=843.46..845.23 rows=708 width=40) (actual time=9.246..9.249 rows=18 loops=1)
   Sort Key: cust_rev.rev DESC
   Sort Method: quicksort  Memory: 25kB
   Buffers: shared hit=258
   CTE cust_rev
     ->  HashAggregate  (cost=677.25..703.78 rows=2123 width=40) (actual time=7.771..8.298 rows=1707 loops=1)
           Group Key: o.customer_id
           Batches: 1  Memory Usage: 881kB
           Buffers: shared hit=258
           ->  Hash Join  (cost=228.36..638.60 rows=7729 width=14) (actual time=1.650..5.903 rows=7917 loops=1)
                 Hash Cond: (oi.order_id = o.order_id)
                 Buffers: shared hit=258
                 ->  Seq Scan on order_items oi  (cost=0.00..361.93 rows=18393 width=14) (actual time=0.005..1.265 rows=18393 loops=1)
                       Buffers: shared hit=178
                 ->  Hash  (cost=192.79..192.79 rows=2845 width=16) (actual time=1.633..1.634 rows=2876 loops=1)
                       Buckets: 4096  Batches: 1  Memory Usage: 167kB
                       Buffers: shared hit=80
                       ->  Bitmap Heap Scan on orders o  (cost=62.34..192.79 rows=2845 width=16) (actual time=0.249..1.313 rows=2876 loops=1)
                             Recheck Cond: ((order_ts >= (now() - '60 days'::interval)) AND (order_status = ANY ('{paid,shipped,delivered}'::text[])))
                             Heap Blocks: exact=70
                             Buffers: shared hit=80
                             ->  Bitmap Index Scan on idx_orders_rev_ts  (cost=0.00..61.62 rows=2845 width=0) (actual time=0.101..0.101 rows=2876 loops=1)
                                   Index Cond: (order_ts >= (now() - '60 days'::interval))
                                   Buffers: shared hit=10
   InitPlan 2
     ->  Aggregate  (cost=53.08..53.09 rows=1 width=8) (actual time=1.221..1.221 rows=1 loops=1)
           ->  CTE Scan on cust_rev cust_rev_1  (cost=0.00..42.46 rows=2123 width=32) (actual time=0.001..0.788 rows=1707 loops=1)
   ->  CTE Scan on cust_rev  (cost=0.00..53.08 rows=708 width=40) (actual time=9.010..9.232 rows=18 loops=1)
         Filter: ((rev)::double precision >= (InitPlan 2).col1)
         Rows Removed by Filter: 1689
         Buffers: shared hit=258
 Planning:
   Buffers: shared hit=12
 Planning Time: 0.771 ms
 Execution Time: 9.307 ms
(35개 행)

skala_db=#
```

▲ 튜닝 후 — 인덱스/재작성 적용 후 실행계획



```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×37
skala_db=# \i q10_result.sql
WITH cust_rev AS (SELECT o.customer_id, sum(oi.line_total) AS rev
   FROM orders o JOIN order_items oi ON oi.order_id=o.order_id
   WHERE o.order_status IN ('paid','shipped','delivered')
     AND o.order_ts >= now() - interval '60 days' GROUP BY o.customer_id)
SELECT customer_id, rev FROM cust_rev
WHERE rev >= (SELECT percentile_cont(0.99) WITHIN GROUP (ORDER BY rev) FROM cust_rev)
ORDER BY rev DESC;
 customer_id |   rev
-------------+----------
          19 | 33828.24
           8 | 30856.78
           9 | 28940.18
          22 | 28859.38
          24 | 26714.13
          18 | 26244.07
          10 | 26027.25
          15 | 25131.91
           4 | 24365.43
          25 | 23359.85
          20 | 22887.52
          12 | 22727.59
           2 | 22555.95
           7 | 22298.63
           6 | 22133.07
          21 | 22049.44
          26 | 21828.22
          29 | 20696.36
(18개 행)

skala_db=#
```

▲ 실행 결과 (튜닝 전후 동일 — 정확성 보존)

개선 — orders 접근이 **Seq Scan → Bitmap Index Scan (idx_orders_rev_ts)** (60일 ≈ 42%로 Q3보다 선택적이라 인덱스 채택).

결과 — 상위 1% = 18명 (고객 19번 33,828 ~ 29번 20,696, 전후 동일)

비즈니스 인사이트 — 최근 60일 기준 **상위 1% = 18명**이 고매출 구간을 형성. 소수 VIP의 최근 활성도가 높아 이탈 방지·전용 혜택의 ROI가 큼.

Q11. 안전 나눗셈 UDF(f_safe_div)로 채널별 AOV 쿼리 재작성

요구사항 — 분모 0이어도 에러 없이 0 반환하는 UDF로 채널별 AOV. 선택 컬럼: channel, revenue, orders, safe_aov.

병목 — orders = **count(DISTINCT order_id)** → GroupAggregate 입력 (channel, order_id) **14,250행 Sort**.

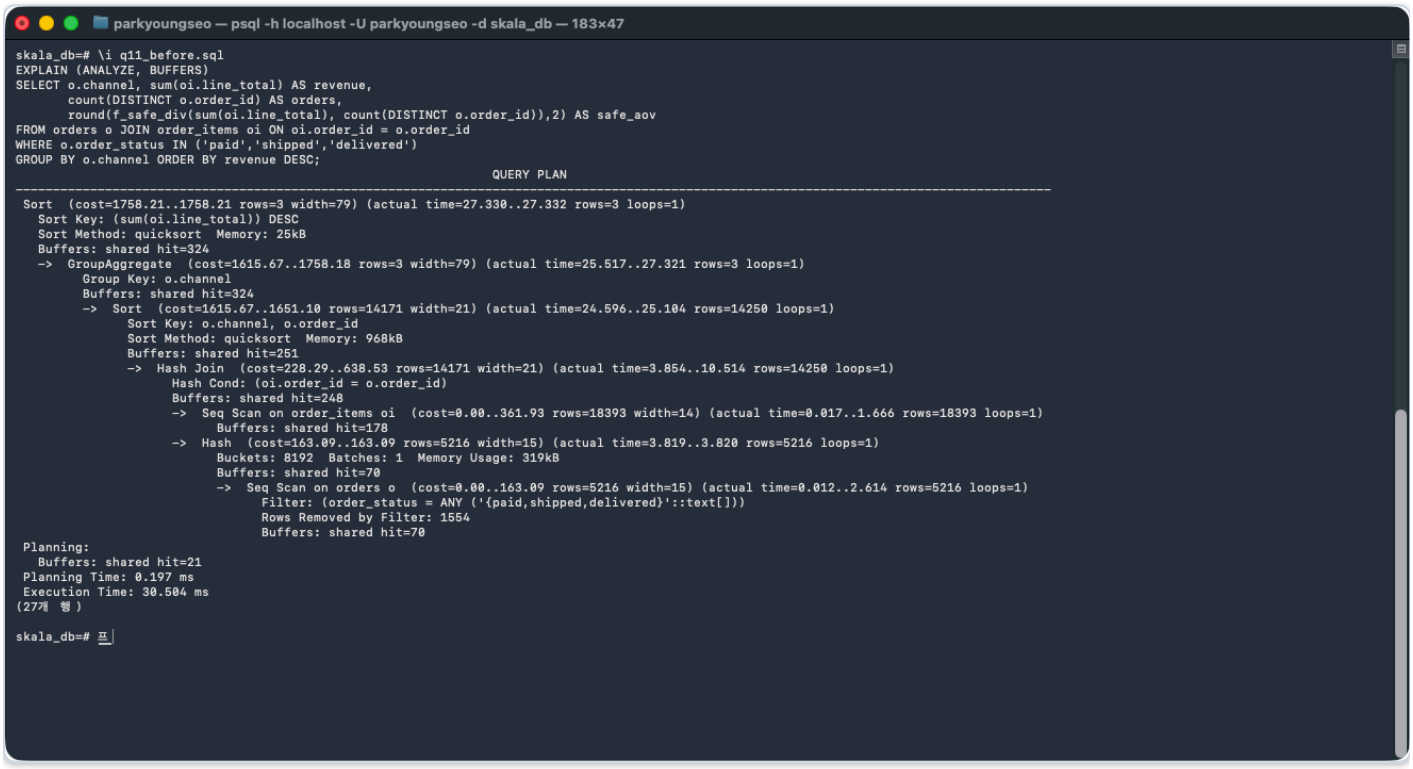
튜닝: 쿼리 재작성 — 주문 단위 선집계 후 채널별 재집계. f_safe_div로 분모 0 안전 처리

SQL — 튜닝 전 쿼리

```
SELECT o.channel, sum(oi.line_total) AS revenue,         -- 채널별 매출
       count(DISTINCT o.order_id) AS orders,              -- 주문수(DISTINCT)
       round(f_safe_div(sum(oi.line_total), count(DISTINCT o.order_id)),2) AS safe_aov  -- 안전 AOV(0나눗셈 방지)
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
WHERE  o.order_status IN ('paid','shipped','delivered')
GROUP BY o.channel ORDER BY revenue DESC;
```

SQL — 튜닝 후 (재작성 쿼리)

```
-- 주문 단위 선집계 후 채널별 재집계 (count(DISTINCT) 제거) + 안전 나눗셈 UDF
SELECT channel, sum(order_rev) AS revenue, count(*) AS orders,  -- 채널별 재집계
       round(f_safe_div(sum(order_rev), count(*)),2) AS safe_aov  -- 안전 AOV(주문수 0이면 0)
FROM (
  SELECT o.order_id, o.channel, sum(oi.line_total) AS order_rev   -- 주문 단위 선집계
  FROM   orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE  o.order_status IN ('paid','shipped','delivered')
  GROUP BY o.order_id, o.channel
) t
GROUP BY channel ORDER BY revenue DESC;
-- f_safe_div(100,0)=0 (에러 없음),  f_safe_div(100,4)=25
```



▲ 튜닝 전 — EXPLAIN (ANALYZE, BUFFERS) baseline 실행계획

```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×49

skala_db=# \i q11_after.sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT channel, sum(order_rev) AS revenue, count(*) AS orders,
       round(f_safe_div(sum(order_rev), count(*)),2) AS safe_aov
FROM (SELECT o.order_id, o.channel, sum(oi.line_total) AS order_rev
      FROM orders o JOIN order_items oi ON oi.order_id=o.order_id
      WHERE o.order_status IN ('paid','shipped','delivered')
      GROUP BY o.order_id, o.channel) t
GROUP BY channel ORDER BY revenue DESC;
                                                                    QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=927.01..927.51 rows=200 width=79) (actual time=15.271..15.273 rows=3 loops=1)
   Sort Key: (sum((sum(oi.line_total)))) DESC
   Sort Method: quicksort  Memory: 25kB
   Buffers: shared hit=248
   ->  HashAggregate  (cost=865.87..919.37 rows=200 width=79) (actual time=15.260..15.265 rows=3 loops=1)
         Group Key: o.channel
         Batches: 1  Memory Usage: 40kB
         Buffers: shared hit=248
         ->  HashAggregate  (cost=709.39..774.59 rows=5216 width=47) (actual time=12.219..14.144 rows=5216 loops=1)
               Group Key: o.order_id
               Batches: 1  Memory Usage: 2257kB
               Buffers: shared hit=248
               ->  Hash Join  (cost=228.29..638.53 rows=14171 width=21) (actual time=2.970..8.438 rows=14250 loops=1)
                     Hash Cond: (oi.order_id = o.order_id)
                     Buffers: shared hit=248
                     ->  Seq Scan on order_items oi  (cost=0.00..361.93 rows=18393 width=14) (actual time=0.007..1.317 rows=18393 loops=1)
                           Buffers: shared hit=178
                     ->  Hash  (cost=163.09..163.09 rows=5216 width=15) (actual time=2.955..2.956 rows=5216 loops=1)
                           Buckets: 8192  Batches: 1  Memory Usage: 306kB
                           Buffers: shared hit=70
                           ->  Seq Scan on orders o  (cost=0.00..163.09 rows=5216 width=15) (actual time=0.019..1.503 rows=5216 loops=1)
                                 Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                                 Rows Removed by Filter: 1554
                                 Buffers: shared hit=70
 Planning:
   Buffers: shared hit=17
 Planning Time: 0.396 ms
 Execution Time: 15.437 ms
(28개 행 )

skala_db=#
```

▲ 튜닝 후 — 인덱스/재작성 적용 후 실행계획

```
parkyoungseo — psql -h localhost -U parkyoungseo -d skala_db — 183×27

skala_db=# \i q11_result.sql
SELECT channel, sum(order_rev) AS revenue, count(*) AS orders,
       round(f_safe_div(sum(order_rev), count(*)),2) AS safe_aov
FROM (SELECT o.order_id, o.channel, sum(oi.line_total) AS order_rev
      FROM orders o JOIN order_items oi ON oi.order_id=o.order_id
      WHERE o.order_status IN ('paid','shipped','delivered')
      GROUP BY o.order_id, o.channel) t
GROUP BY channel ORDER BY revenue DESC;
   channel   |  revenue   | orders | safe_aov
-------------+------------+--------+----------
 mobile      | 2404315.94 |   2633 |   913.15
 web         | 1249981.97 |   1310 |   954.05
 marketplace | 1166079.97 |   1273 |   916.01
(3개 행 )

SELECT f_safe_div(100,0) AS div_by_zero, f_safe_div(100,4) AS normal_div;
 div_by_zero |     normal_div
-------------+--------------------
           0 | 25.000000000000000
(1개 행 )

skala_db=#
```

▲ 실행 결과 (튜닝 전후 동일 — 정확성 보존)

개선 — count(DISTINCT)가 유발한 **이중 Sort(14,250행 + 외부 정렬) → HashAggregate**. 캡처 실행시간도 절반 수준으로 감소 (30.5ms → 15.4ms).

결과 — mobile 240만/2,633/913 · web 125만/1,310/954 · marketplace 117만/1,273/916 (전후 동일)

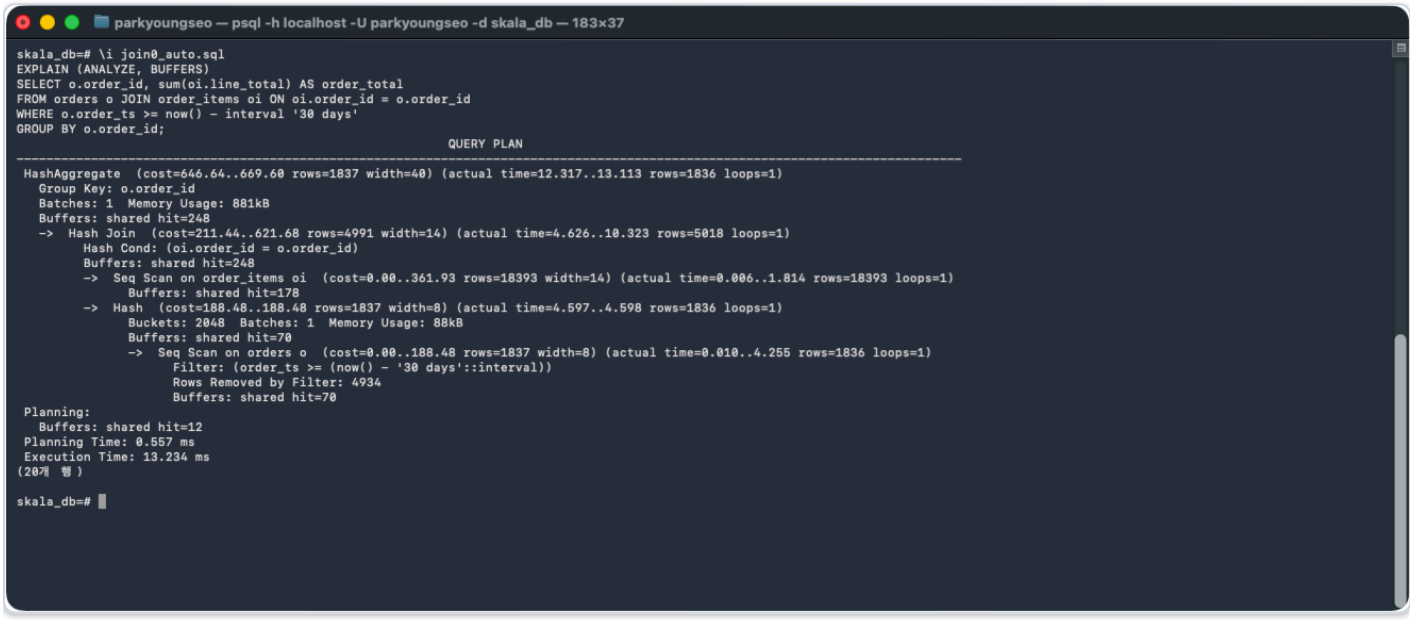
비즈니스 인사이트 — **mobile이 매출 최다(240만)**이나 채널별 AOV는 web(954)>marketplace(916)>mobile(913)로 근소. 유입이 모바일에 몰리므로 모바일 UX·전환율 개선의 레버리지가 가장 큼.

제출물 2 · 3가지 조인 알고리즘 비교

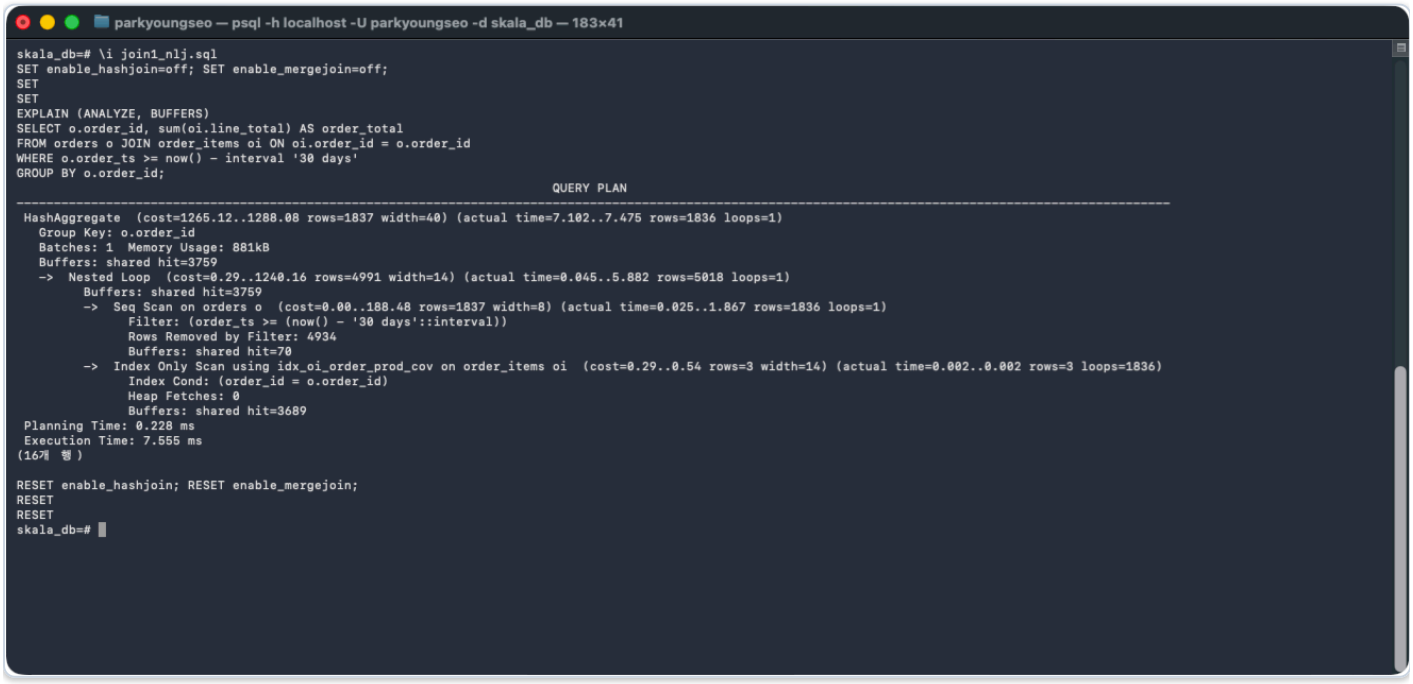
동일 쿼리(최근 30일 주문 × order_items 집계)를 `enable_*` 플래그로 각 조인 전략을 강제하여 실행계획·특성을 비교한다. 외부(orders) 1,836행 × 내부(order_items) 18,393행.

전략	실행계획 핵심	실행시간	유리한 상황
Nested Loop	외부 각 행마다 내부를 **Index Only Scan**으로 프로브 (loops=1,836, Heap Fetches 0)	약 7.6ms	외부가 작고 내부에 인덱스가 있을 때. 외부가 커지면 프로브 폭증에 불리.
Hash Join	orders로 해시 빌드 + order_items 전량 Seq Scan 후 매칭 (옵티마이저 기본 선택)	약 11.5ms	인덱스 없거나 대량 등가 조인에 안정적.
Merge Join	order_items는 인덱스 정렬 스캔, orders는 Sort 후 **병합**	약 7.8ms	양쪽이 이미 조인 키로 정렬되어 있을 때 매우 효율.

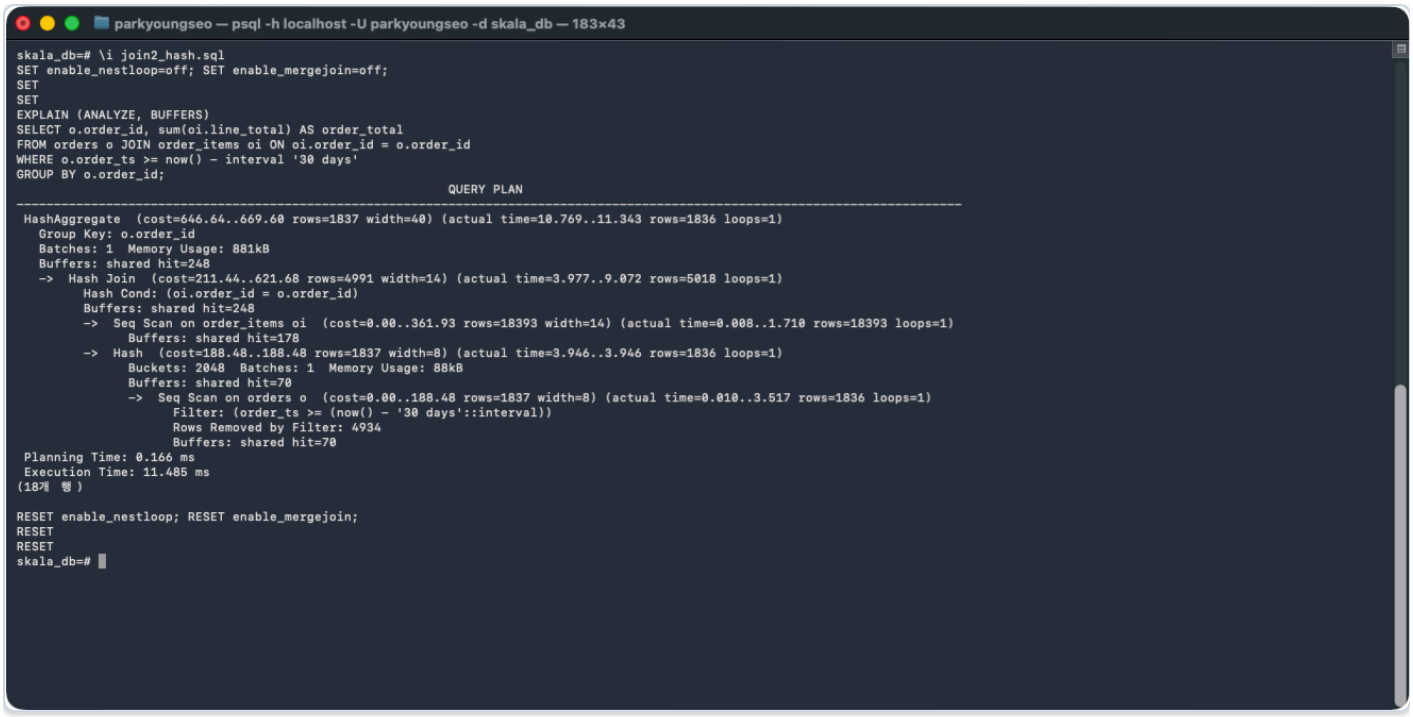
※ 위 실행시간은 각 캡처(강제 실행)의 EXPLAIN Execution Time이다. 단일 측정이라 편차가 커 **알고리즘 우열의 근거가 못 된다** — 이 회차에선 강제 Hash가 11.5ms로 가장 높게 나왔지만 이는 계측 노이즈이며, 옵티마이저는 비용 추정상 **Hash Join**을 기본 선택한다(자동 선택 캡처). 핵심은 각 전략의 **계획 구조와 적용 상황**이다.



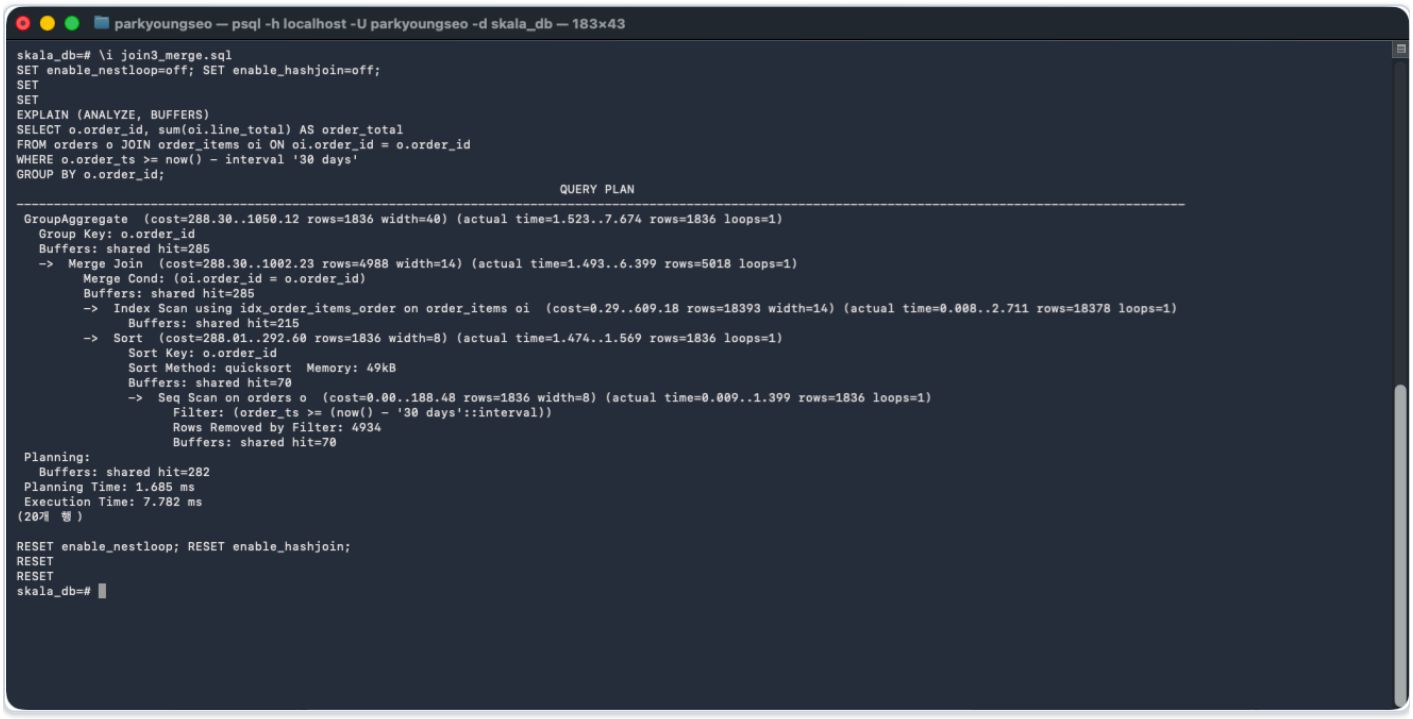
▲ [기본] 옵티마이저 자동 선택 → Hash Join 채택 (자동 선택 데모)



▲ [1] Nested Loop 강제 — Index Only Scan 프로브(loops=1,836)



▲ [2] Hash Join 강제 — order_items 전량 스캔 + 해시



▲ [3] Merge Join 강제 — Sort 후 병합

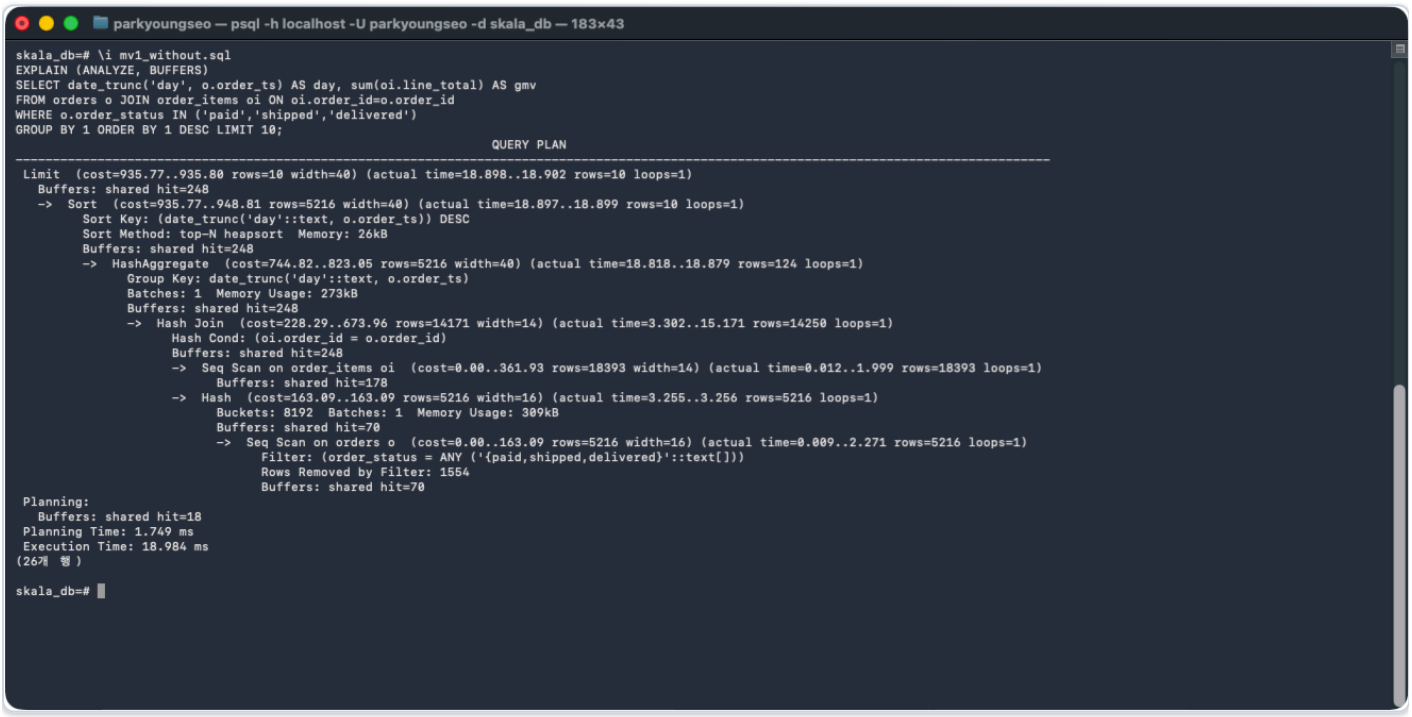
제출물 3 · Materialized View (mv_daily_gmv)

매일 총 판매금액(GMV) 리포트를 **매번 JOIN+SUM** 하면 느리므로, 사전 집계한 `mv_daily_gmv` 로 리포트 질의를 가속한다.

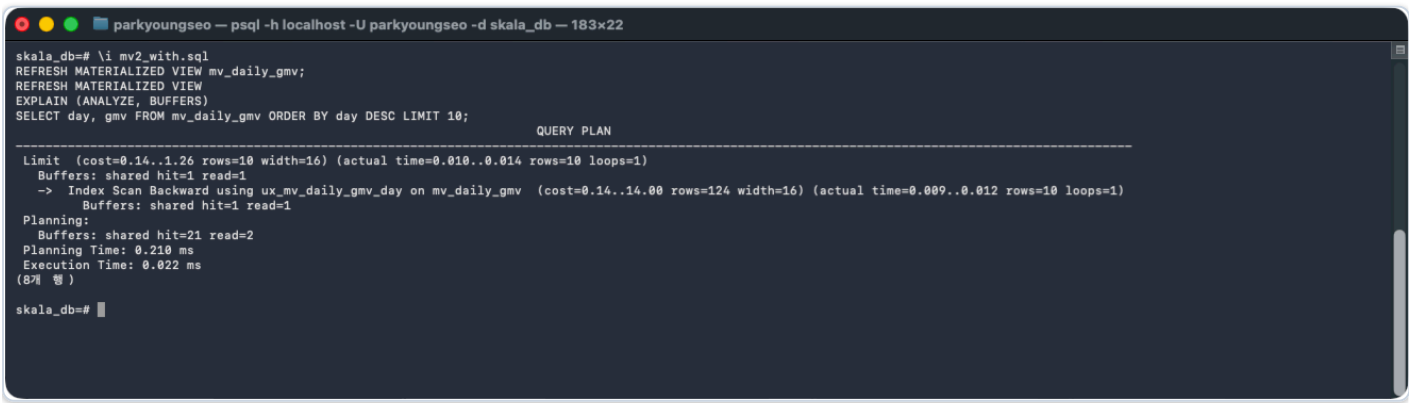
```
CREATE MATERIALIZED VIEW mv_daily_gmv AS
SELECT date_trunc('day', o.order_ts) AS day, sum(oi.line_total) AS gmv
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY 1;
CREATE UNIQUE INDEX ux_mv_daily_gmv_day ON mv_daily_gmv(day);  -- CONCURRENTLY 갱신 대비
```

구분	실행계획	실행시간
MV 미사용 (JOIN+SUM 직접 집계)	Seq Scan×2 + Hash Join + HashAggregate (14,250행 집계)	약 **19ms**
MV 사용 (사전 집계 124행 조회)	Index Scan Backward (ux_mv_daily_gmv_day)	약 **0.02ms**

개선 — 캡처 EXPLAIN 기준 약 **860배** 단축(19ms→0.02ms). 세 제출물 중 **유일하게 대규모의 실질 시간 이득**으로, 집계 자체를 미리 계산해 두므로 order_items 전량 스캔·집계를 통째로 건너뛴다. 리포트성(반복) 질의에 Materialized View가 효과적. 비용: 저장공간 + 갱신(REFRESH) 필요. **갱신 전략**: 리포트가 오후 3시 기준이면 매일 15:00 스케줄러(cron/pg_cron)로 `REFRESH MATERIALIZED VIEW` ; 무중단이 필요하면 UNIQUE 인덱스 + `REFRESH ... CONCURRENTLY` .



▲ [MV 미사용] 일자별 GMV 직접 집계 — JOIN+SUM (Hash Join + HashAggregate), Execution 약 19ms



▲ [MV 사용] REFRESH 후 같은 리포트를 MV에서 조회 — Index Scan Backward, Execution 약 0.02ms

▲ MV 결과 미리보기 (일자별 GMV) 및 행 수(124일)

종합 요약

#	튜닝 기법	계획 변화 (전 → 후) — 핵심 근거	EXPLAIN 실행시간 (캡처, 전→후)
Q1	부분 인덱스	Seq Scan → Bitmap Index Scan (orders 접근 인덱스화)	8.6 → 7.5ms
Q2	쿼리 재작성	GroupAggregate+Sort(14,250행) → HashAggregate	21.1 → 19.8ms
Q3	계획 분석	Seq Scan 유지 (90일 저선택도, 옵티마이저 판단 정당)	12.7 → 13.9ms (계획 동일)
Q4	계획 분석	전량 집계·RANK Sort 불가피 → 전처리(MV)가 정답	14.2 → 18.5ms (계획 동일)
Q5	쿼리 재작성	GroupAggregate+Sort(14,250행) → HashAggregate	14.2 → 18.3ms *
Q6	복합 부분 인덱스	Index Scan(heap 재확인, buf 5,539) → Index Only Scan	6.0 → 10.0ms * (버퍼 5,539→4,460)
Q7	부분 인덱스	Seq Scan(600) → Bitmap Index Scan(61) — 접근행수↓	0.21 → 0.17ms *
Q8	커버링 인덱스	Seq Scan → Index Only Scan (Heap Fetches 0)	2.3 → 1.5ms
Q9	쿼리 재작성	GroupAggregate+Sort(5,216행) → HashAggregate	21.8 → 18.0ms
Q10	부분 인덱스	Seq Scan → Bitmap Index Scan (60일≈42%)	10.7 → 9.3ms
Q11	쿼리 재작성+UDF	이중 Sort(14,250행+외부) → HashAggregate	30.5 → 15.4ms
조인	NLJ/Hash/Merge	전략별 실행계획·특성 비교	NLJ 7.6·Hash 11.5·Merge 7.8ms
MV	사전 집계	JOIN+SUM(전량 집계) → Index Scan (MV 124행)	**19 → 0.02ms (~860배)**

※ **측정 유의점** — 데이터 규모(orders 6,770·order_items 18,393)가 작아 절대 실행시간은 수 ms대이고, EXPLAIN ANALYZE의 행별 계측 오버헤드로 **단일 실행 시간은 실행마다 편차가 크다**. 위 표의 시간은 각 문항 캡처(EXPLAIN ANALYZE)의 **Execution Time 전→후** 값이다. 이 편차 때문에 ***표시 문항은 튜닝 후 단일 측정을 곧이곧대로 읽으면 안 된다** — Q5·Q6은 후가 오히려 근소히 높게 나왔지만(노이즈) 계획·버퍼는 개선(Q5 대형 Sort 제거, Q6 버퍼 5,539→4,460·Heap Fetches 0), Q7은 강제 인덱스(후)가 근소히 빨랐으나 옵티마이저는 비용상 Seq Scan을 유지(접근 행수 600→61). 따라서 튜닝의 핵심 근거는 시간이 아니라 **실행계획 노드(스캔·조인·정렬)와 접근 행수·버퍼의 변화**이며, 이 효과는 데이터가 커질수록 실질 이득으로 확대된다. **인덱스가 항상 이득은 아니며**(Q3·Q4는 Seq Scan 유지가 최적, Q7은 소형 테이블이라 인덱스 이득이 측정 노이즈 수준 — 옵티마이저는 비용상 Seq Scan 유지), 비용기반 옵티마이저의 선택을 함께 분석하였다. 실질적으로 큰 시간 이득은 **Materialized View(약 860배)**에서 나온다.

스캔·인덱스·조인 전략 정리

- **스캔 방식** — **Seq Scan**(소형·저선택도 필터: Q3·Q4·Q7·Q8에서 옵티마이저 채택), **Bitmap Index Scan**(부분 인덱스로 선택적 구간만: Q1·Q10), **Index Only Scan**(커버링/복합 인덱스로 heap 접근 제거, Heap Fetches 0: Q6·Q8)를 상황별로 확인.
- **인덱스 전략** — **부분 인덱스**(매출 상태·품절 조건만 색인: Q1·Q7·Q10), **복합 부분 인덱스**((customer_id, order_ts)+status: Q6), **커버링 인덱스**(product_id INCLUDE rating: Q8). 인덱스 크기·유지비를 낮추면서 필요한 행만 겨냥.
- **쿼리 재작성** — `count(DISTINCT)`가 유발하는 대형 Sort를 **주문 단위 선집계 → HashAggregate**로 제거(Q2·Q5·Q9·Q11). 인덱스 없이 계획 자체를 개선.
- **조인 전략** — 이 데이터에선 대량 등가 조인에 **Hash Join**이 기본 선택. 외부가 작고 내부에 인덱스가 있으면 **Nested Loop**(Index Only Scan 프로브), 정렬된 입력이면 **Merge Join**이 유리(제출물 2 참조).
- **사전 집계** — 반복 조회되는 리포트성 질의는 **Materialized View**로 집계를 미리 계산해 전량 스캔·집계를 통째로 회피(제출물 3, 약 860배).

부록 · DB 엔진별 옵티마이저 특징 비교

같은 SQL이라도 엔진마다 옵티마이저 동작·튜닝 접근이 다르다. 본 실습(PostgreSQL) 관점에서 주요 엔진을 정리한다.

엔진	옵티마이저 특징	튜닝 관점
PostgreSQL (본 실습)	통계 기반의 엄격한 **비용기반 옵티마이저(CBO)**. Hash Join·Bitmap Heap Scan·부분/커버링 인덱스 등 분석(OLAP)에 강점.	`EXPLAIN (ANALYZE, BUFFERS)` 로 계획·버퍼 분석, 부분/커버링 인덱스·Materialized View 적극 활용.
MySQL	전통적으로 OLTP 중심이라 **Nested Loop** 의존이 컸으나 8.0부터 **Hash Join** 지원으로 대량 집계 성능 개선.	커버링 인덱스·복합 인덱스 설계가 핵심. 옵티마이저 힌트 제한적.
Oracle	가장 방대·정교한 통계와 **튜닝 힌트(Hint)**. DBA가 실행계획을 세밀하게 제어 가능(상용 강자).	Hint·SQL Plan Baseline으로 계획 고정, 파티셔닝·병렬 활용.
SQL Server	고도로 자동화된 옵티마이저. 실행계획 분석 시 **Missing Index(누락 인덱스) 추천**, CPU 병렬 처리에 강점.	추천 인덱스 검토 + 실행계획 캐시(Plan Cache) 관리.

※ 공통 원리는 동일하다 — **필요한 행만 읽고(스캔·인덱스), 불필요한 정렬을 줄이며(재작성), 반복 집계는 미리 계산(MV)**. 엔진별 차이는 '어떤 계획을 어떻게 강제·유도하느냐'의 방법론 차이다.